В поисках понятного алгоритма консенсуса

Расширенная версия, 2014

Диего Онгаро и Джон Оустерхаут, Стэнфордский университет

Перевод на русский язык. Рисунки и надписи внутри них сохранены в оригинале.

Аннотация

Raft — это консенсусный алгоритм управления реплицируемым журналом. Он дает результат, эквивалентный (мульти-)Paxos, и он так же эффективен, как Paxos, но его структура отличается от Paxos; это делает Raft более понятным, чем Paxos, а также обеспечивает лучшую основу для построения практических систем. Чтобы повысить понятность, Raft разделяет ключевые элементы консенсуса, такие как выборы лидера, репликация журналов и безопасность, а также обеспечивает более высокую степень согласованности, чтобы уменьшить количество состояний, которые необходимо учитывать. Результаты исследования с участием пользователей показывают, что учащимся легче изучить Raft, чем Paxos. Raft также включает новый механизм изменения членства в кластере, который использует перекрывающееся большинство для обеспечения безопасности.

1 Введение

Алгоритмы консенсуса позволяют совокупности машин работать как сплоченная группа, способная пережить сбои некоторых ее членов. По этой причине они играют ключевую роль в создании надежных крупномасштабных программных систем. Paxos доминировал в обсуждении алгоритмов консенсуса в течение последнего десятилетия: большинство реализаций консенсуса основаны на Paxos или находятся под его влиянием, а Paxos стал основным средством, используемым для обучения студентов консенсусу.

К сожалению, Paxos довольно сложно понять, несмотря на многочисленные попытки сделать его более доступным. Более того, его архитектура требует сложных изменений для поддержки практических систем. В результате с Paxos сталкиваются как разработчики систем, так и студенты.

После борьбы с Paxos мы решили найти новый алгоритм консенсуса, который мог бы обеспечить лучшую основу для построения и обучения системы. Наш подход был необычен тем, что нашей основной целью была *понятность*: можем ли мы определить алгоритм консенсуса для практических систем и описать его так, чтобы его было значительно легче изучить, чем Paxos? Более того, мы хотели, чтобы алгоритм способствовал развитию интуиции, необходимой для разработчиков систем. Важно было не просто чтобы алгоритм работал, но и чтобы было очевидно, почему он работает.

Результатом этой работы является алгоритм консенсуса под названием Raft. При разработке Raft мы применили специальные методы для повышения понятности, включая декомпозицию (Raft разделяет выбор лидера, репликацию журналов и безопасность) и сокращение пространства состояний (по сравнению с Paxos, Raft уменьшает степень недетерминированности и вероятность несогласованности серверов друг с другом). Исследование пользователей с участием 43 студентов двух университетов показывает, что Raft значительно легче понять, чем Paxos: после изучения обоих алгоритмов 33 из этих студентов смогли ответить на вопросы о Raft лучше, чем на вопросы о Paxos.

Raft во многом похож на существующие консенсусные алгоритмы (в первую очередь, на Viewstamped Replication Оки и Лискова), но имеет несколько новых особенностей:

- **Сильный лидер**: Raft использует более сильную форму лидерства, чем другие алгоритмы консенсуса. Например, записи журнала передаются только от лидера на другие серверы. Это упрощает управление реплицируемым журналом и облегчает понимание Raft.
- **Выборы лидера**: Raft использует случайные таймеры для выбора лидеров. Это добавляет лишь небольшое количество механизмов к heartbeat-сообщениям, уже необходимым для любого алгоритма консенсуса, при этом разрешая конфликты просто и быстро.
- **Изменения членства**: механизм Raft для изменения набора серверов в кластере использует новый подход *совместного консенсуса*, при котором большинство двух разных конфигураций перекрываются во время переходов. Это позволяет кластеру продолжать нормально работать во время изменений конфигурации.

Мы считаем, что Raft превосходит Paxos и другие алгоритмы консенсуса как в образовательных целях, так и в качестве основы для реализации. Он проще и понятнее других алгоритмов; он описан достаточно полно, чтобы удовлетворить потребности практической системы; он имеет несколько реализаций с открытым исходным кодом и используется несколькими компаниями; его характеристики безопасности официально определены и доказаны; и его эффективность сравнима с другими алгоритмами.

Оставшаяся часть статьи знакомит с проблемой репликации конечного автомата (раздел 2), обсуждает сильные и слабые стороны Paxos (раздел 3), описывает наш общий подход к понятности (раздел 4), представляет алгоритм консенсуса Raft (разделы 5–8), оценивает Raft (раздел 9) и обсуждает связанную работу (раздел 10).

2 Реплицированные конечные автоматы

Алгоритмы консенсуса обычно возникают в контексте *реплицируемых конечных автоматов*. При таком подходе конечные автоматы на наборе серверов вычисляют идентичные копии одного и того же состояния и могут продолжать работу, даже если некоторые из серверов не работают. Реплицированные конечные автоматы используются для решения различных проблем отказоустойчивости в распределенных системах. Например, крупномасштабные системы с одним лидером кластера, такие как GFS, HDFS и RAMCloud, обычно используют отдельный реплицируемый конечный автомат для управления выборами лидера и хранения информации о конфигурации, которая должна пережить сбой лидера. Примеры реплицируемых конечных автоматов включают Chubby и ZooKeeper.

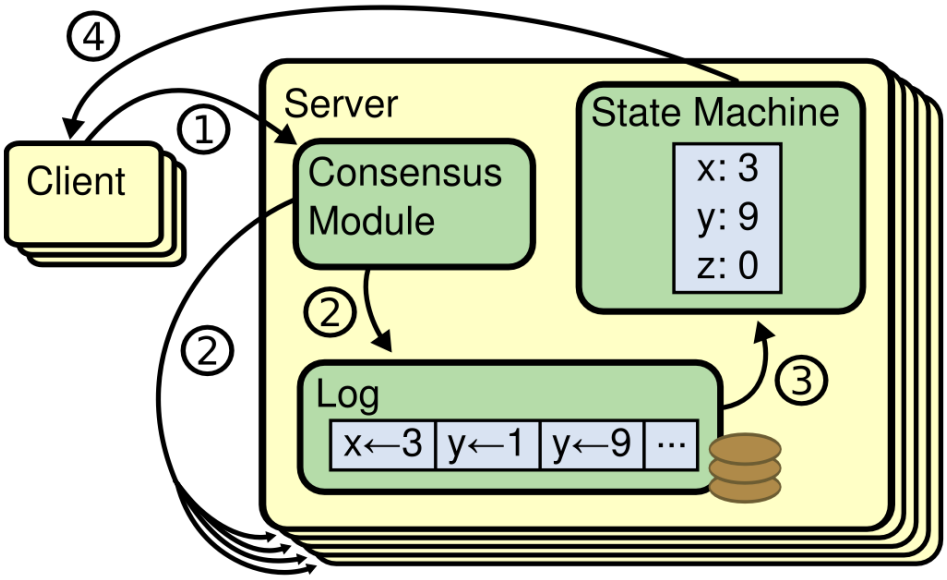


Figure 1: Replicated state machine architecture. The consensus algorithm manages a replicated log containing state machine commands from clients. The state machines process identical sequences of commands from the logs, so they produce the same outputs.

Реплицированные конечные автоматы обычно реализуются с использованием реплицированного журнала, как показано на рисунке 1. Каждый сервер хранит журнал, содержащий ряд команд, которые его конечный автомат выполняет по порядку. Каждый журнал содержит одни и те же команды в одном и том же порядке, поэтому каждый конечный автомат обрабатывает одну и ту же последовательность команд. Поскольку конечные автоматы детерминированы, каждый из них вычисляет одно и то же состояние и одну и ту же последовательность выходных данных.

Поддержание согласованности реплицированного журнала — задача алгоритма консенсуса. Модуль консенсуса на сервере получает команды от клиентов и добавляет их в свой журнал. Он взаимодействует с модулями консенсуса на других серверах, чтобы гарантировать, что каждый журнал в конечном итоге будет содержать одни и те же запросы в одном и том же порядке, даже если некоторые серверы выходят из строя. Как только команды правильно реплицируются, конечный автомат каждого сервера обрабатывает их в порядке журнала, а выходные данные возвращаются клиентам. В результате серверы образуют единый высоконадежный конечный автомат.

Алгоритмы консенсуса для практических систем обычно обладают следующими свойствами:

- Они обеспечивают *безопасность* (никогда не возвращают неправильный результат) при всех условиях без византийских отказов, включая сетевые задержки, разделение сети, потерю пакетов, дублирование и переупорядочение.
- Они полностью функциональны (*доступны*), пока большинство серверов находятся в рабочем состоянии и могут взаимодействовать друг с другом и с клиентами. Таким образом, типичный кластер из пяти серверов может допустить сбой любых двух серверов. Предполагается, что серверы выходят из строя при остановке; позже они могут восстановиться из состояния стабильного хранилища и снова присоединиться к кластеру.
- Они не зависят от времени для обеспечения согласованности журналов: сбои в работе часов и чрезмерные задержки сообщений могут в худшем случае вызвать проблемы с доступностью.

- В обычном случае команда может завершиться, как только большая часть кластера ответит на один раунд вызовов удаленных процедур; меньшинство медленных серверов не обязательно влияет на общую производительность системы.

3 Что не так с Paxos?

За последние десять лет протокол Лесли Лампорта Paxos стал почти синонимом консенсуса: это протокол, который чаще всего преподают на курсах, и большинство реализаций консенсуса используют его в качестве отправной точки. Paxos сначала определяет протокол, позволяющий достичь соглашения по единому решению, например, по одной реплицируемой записи журнала. Мы называем это подмножество *одноразовым Paxos*. Paxos затем объединяет несколько экземпляров этого протокола для облегчения принятия ряда решений, таких как журнал (*multi-Paxos*). Paxos обеспечивает безопасность и живучесть, а также поддерживает изменения в членстве в кластере. Его корректность доказана, и в нормальном случае она эффективна.

К сожалению, у Paxos есть два существенных недостатка. Первый недостаток заключается в том, что Paxos исключительно труден для понимания. Полное объяснение заведомо неясно; Понять это удается немногим людям, и только с большим усилием. В результате было предпринято несколько попыток объяснить Paxos более простыми словами. Эти объяснения сосредоточены на варианте для одного решения, но они по-прежнему сложны. В ходе неофициального опроса участников NSDI 2012 мы обнаружили мало людей, которым был бы удобен Paxos, даже среди опытных исследователей. Мы сами боролись с Paxos; мы не смогли понять весь протокол до тех пор, пока не прочитали несколько упрощенных объяснений и не разработали собственный альтернативный протокол — процесс, который занял почти год.

Мы предполагаем, что непрозрачность Paxos обусловлена выбором подмножества с одним решением в качестве основы. Одноразовый Paxos плотен и тонок: он разделен на два этапа, которые не имеют простых интуитивных объяснений и не могут быть поняты самостоятельно. Из-за этого трудно понять, почему протокол одноразового Paxos работает. Правила композиции для multi-Paxos добавляют значительную сложность и тонкость. Мы считаем, что общую проблему достижения консенсуса по множественным решениям (т. е. журнала вместо одной записи) можно разложить другими способами, более прямыми и очевидными.

Вторая проблема с Paxos заключается в том, что он не обеспечивает хорошей основы для создания практических реализаций. Одна из причин заключается в том, что для multi-Paxos не существует общепринятого алгоритма. Описания Лэмпорта в основном касаются одноразового Paxos; он набросал возможные подходы к multi-Paxos, но многие детали отсутствуют. Было несколько попыток конкретизировать и оптимизировать Paxos, например [26], [39] и [13], но они отличаются друг от друга и от эскизов Лэмпорта. Такие системы, как Chubby, реализовали алгоритмы, подобные Paxos, но в большинстве случаев их подробности не публиковались.

Более того, архитектура Paxos не подходит для построения практических систем; это еще одно следствие декомпозиции по одному решению. Например, мало пользы от независимого выбора набора записей журнала и последующего объединения их в последовательный журнал; это только добавляет сложности. Проще и эффективнее спроектировать систему на основе журнала, в котором новые записи добавляются последовательно в ограниченном порядке. Другая проблема заключается в том, что Paxos в своей основе использует симметричный одноранговый подход (хотя в конечном итоге он предполагает слабую форму лидерства в качестве оптимизации производительности). Это имеет смысл в упрощенном мире, где будет приниматься только одно решение, но этот подход используется немногими практическими системами. Если необходимо принять ряд решений, проще и быстрее сначала выбрать лидера, а затем поручить ему координировать решения.

В результате практические системы мало похожи на Paxos. Каждая реализация начинается с Paxos, выявляются трудности ее реализации, а затем разрабатывается существенно другая архитектура. Это отнимает много времени и чревато ошибками, а трудности понимания Paxos усугубляют проблему. Формулировка Paxos может быть хорошей для доказательства теорем о ее корректности, но реальные реализации настолько отличаются от Paxos, что доказательства не имеют особой ценности. Типичным

является следующий комментарий разработчиков Chubby:

> Между описанием алгоритма Paxos и потребностями реальной системы существуют значительные пробелы. . . . окончательная система будет основана на недоказанном протоколе.

Из-за этих проблем мы пришли к выводу, что Paxos не обеспечивает хорошей основы ни для построения системы, ни для обучения. Учитывая важность консенсуса в крупномасштабных программных системах, мы решили посмотреть, сможем ли мы разработать альтернативный алгоритм консенсуса с лучшими свойствами, чем Paxos. Raft — результат этого эксперимента.

4 Проектирование для понятности

При разработке Raft у нас было несколько целей: он должен обеспечить полную и практичную основу для построения системы, чтобы значительно сократить объем проектной работы, требуемой от разработчиков; оно должно быть безопасным при любых условиях и доступным в типичных условиях эксплуатации; и он должен быть эффективен для общих операций. Но нашей самой важной целью и самой сложной задачей была *понятность*. Большая аудитория должна легко понять алгоритм. Кроме того, должна быть возможность получить представление об алгоритме, чтобы разработчики систем могли создавать расширения, которые неизбежны в реальных реализациях.

При разработке Raft было множество моментов, где нам приходилось выбирать между альтернативными подходами. В этих ситуациях мы оценивали альтернативы на основе понятности: насколько сложно объяснить каждую альтернативу (например, насколько сложно ее пространство состояний и имеет ли она тонкие последствия?) и насколько легко читателю будет полностью понять подход и его последствия?

Мы признаем, что такой анализ имеет высокую степень субъективности; тем не менее, мы использовали два метода, которые обычно применимы. Первый метод — это хорошо известный подход декомпозиции проблем: везде, где это возможно, мы делили проблемы на отдельные части, которые можно было решить, объяснить и понять относительно независимо. Например, в Raft мы разделили выборы лидера, репликацию журналов, безопасность и изменения членства.

Наш второй подход заключался в том, чтобы упростить пространство состояний за счет сокращения количества рассматриваемых состояний, сделать систему более последовательной и устранить недетерминизм, где это возможно. В частности, в журналах не допускается наличие дыр, а Raft ограничивает способы, которыми журналы могут стать несовместимыми друг с другом. Хотя в большинстве случаев мы пытались устранить недетерминизм, в некоторых ситуациях недетерминизм действительно улучшает понятность. В частности, рандомизированные подходы вводят недетерминизм, но они имеют тенденцию уменьшать пространство состояний, обрабатывая все возможные варианты одинаковым образом («выберите любой; это не имеет значения»). Мы использовали рандомизацию, чтобы упростить алгоритм выборов лидера Raft.

5 Алгоритм консенсуса Raft

Raft — это алгоритм управления реплицируемым журналом формы, описанной в разделе 2. На рисунке 2 в кратком виде представлен алгоритм для справки, а на рисунке 3 перечислены ключевые свойства алгоритма; элементы этих фигур обсуждаются по частям в оставшейся части этого раздела.

State

Persistent state on all servers:
(Updated on stable storage before responding to RPCs)

currentTerm	latest term server has seen (initialized to 0 on first boot, increases monotonically)
votedFor	candidateId that received vote in current term (or null if none)
log[]	log entries; each entry contains command for state machine, and term when entry was received by leader (first index is 1)

Volatile state on all servers:

commitIndex	index of highest log entry known to be committed (initialized to 0, increases monotonically)
lastApplied	index of highest log entry applied to state machine (initialized to 0, increases monotonically)

Volatile state on leaders:
(Reinitialized after election)

nextIndex[]	for each server, index of the next log entry to send to that server (initialized to leader last log index + 1)
matchIndex[]	for each server, index of highest log entry known to be replicated on server (initialized to 0, increases monotonically)

AppendEntries RPC

Invoked by leader to replicate log entries (§5.3); also used as heartbeat (§5.2).

Arguments:

term	leader's term
leaderId	so follower can redirect clients
prevLogIndex	index of log entry immediately preceding new ones
prevLogTerm	term of prevLogIndex entry
entries[]	log entries to store (empty for heartbeat; may send more than one for efficiency)
leaderCommit	leader's commitIndex

Results:

term	currentTerm, for leader to update itself
success	true if follower contained entry matching prevLogIndex and prevLogTerm

Receiver implementation:

1. Reply false if term < currentTerm (§5.1)
2. Reply false if log doesn't contain an entry at prevLogIndex whose term matches prevLogTerm (§5.3)
3. If an existing entry conflicts with a new one (same index but different terms), delete the existing entry and all that follow it (§5.3)
4. Append any new entries not already in the log
5. If leaderCommit > commitIndex, set commitIndex = min(leaderCommit, index of last new entry)

RequestVote RPC

Invoked by candidates to gather votes (§5.2).

Arguments:

term	candidate's term
candidateId	candidate requesting vote
lastLogIndex	index of candidate's last log entry (§5.4)
lastLogTerm	term of candidate's last log entry (§5.4)

Results:

term	currentTerm, for candidate to update itself
voteGranted	true means candidate received vote

Receiver implementation:

1. Reply false if term < currentTerm (§5.1)
2. If votedFor is null or candidateId, and candidate's log is at least as up-to-date as receiver's log, grant vote (§5.2, §5.4)

Rules for Servers

All Servers:

- If commitIndex > lastApplied: increment lastApplied, apply log[lastApplied] to state machine (§5.3)
- If RPC request or response contains term T > currentTerm: set currentTerm = T, convert to follower (§5.1)

Followers (§5.2):

- Respond to RPCs from candidates and leaders
- If election timeout elapses without receiving AppendEntries RPC from current leader or granting vote to candidate: convert to candidate

Candidates (§5.2):

- On conversion to candidate, start election:
  - Increment currentTerm
  - Vote for self
  - Reset election timer
  - Send RequestVote RPCs to all other servers
- If votes received from majority of servers: become leader
- If AppendEntries RPC received from new leader: convert to follower
- If election timeout elapses: start new election

Leaders:

- Upon election: send initial empty AppendEntries RPCs (heartbeat) to each server; repeat during idle periods to prevent election timeouts (§5.2)
- If command received from client: append entry to local log, respond after entry applied to state machine (§5.3)
- If last log index ≥ nextIndex for a follower: send AppendEntries RPC with log entries starting at nextIndex
  - If successful: update nextIndex and matchIndex for follower (§5.3)
  - If AppendEntries fails because of log inconsistency: decrement nextIndex and retry (§5.3)
- If there exists an N such that N > commitIndex, a majority of matchIndex[i] ≥ N, and log[N].term == currentTerm: set commitIndex = N (§5.3, §5.4).

Figure 2: A condensed summary of the Raft consensus algorithm (excluding membership changes and log compaction). The server behavior in the upper-left box is described as a set of rules that trigger independently and repeatedly. Section numbers such as §5.2 indicate where particular features are discussed. A formal specification [31] describes the algorithm more precisely.

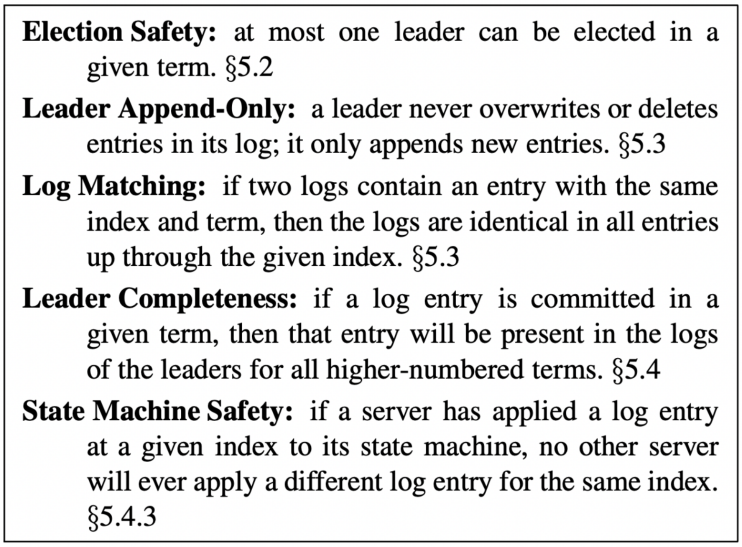


Figure 3: Raft guarantees that each of these properties is true at all times. The section numbers indicate where each property is discussed.

Raft реализует консенсус, сначала выбирая выдающегося *лидера*, а затем наделяя его полной ответственностью за управление реплицируемым журналом. Лидер принимает записи журнала от клиентов, реплицирует их на другие серверы и сообщает серверам, когда безопасно применять записи журнала к их конечным машинам. Наличие лидера упрощает управление реплицируемым журналом. Например, лидер может решить, где разместить новые записи в журнале, не консультируясь с другими серверами, и данные простым способом передаются от лидера к другим серверам. Лидер может выйти из строя или отключиться от других серверов, и в этом случае выбирается новый лидер.

Учитывая лидерский подход, Raft разбивает проблему консенсуса на три относительно независимые подзадачи, которые обсуждаются в следующих подразделах:

- Выборы лидера: новый лидер должен быть выбран, если существующий лидер терпит неудачу (раздел 5.2).
- Репликация журналов: лидер должен принимать записи журналов от клиентов и реплицировать их по кластеру, заставляя другие журналы согласовываться с его собственными (раздел 5.3).
- Безопасность: ключевым свойством безопасности для Raft является свойство State Machine Safety на рисунке 3: если какой-либо сервер применил определенную запись журнала к своему конечному автомату, то никакой другой сервер не может применить другую команду для того же индекса журнала. В разделе 5.4 описано, как Raft обеспечивает это свойство; решение предполагает дополнительное ограничение на механизм выборов, описанный в разделе 5.2.

После представления алгоритма консенсуса в этом разделе обсуждаются вопросы доступности и роли времени в системе.

5.1 Основы Raft

Кластер Raft содержит несколько серверов; пять — типичное число, позволяющее системе выдержать два сбоя. В любой момент времени каждый сервер находится в одном из трех состояний: *лидер*, *ведомый* или *кандидат*. При нормальной работе есть ровно один лидер, а все остальные серверы являются его ведомыми. Ведомые пассивны: они не подают никаких запросов самостоятельно, а просто отвечают на запросы лидеров и кандидатов. Лидер обрабатывает все запросы клиентов (если клиент обращается к ведомому серверу, ведомый перенаправляет его лидеру). Третье состояние, кандидат, используется для избрания нового лидера, как описано в разделе 5.2. На рис. 4 показаны состояния и их переходы; переходы обсуждаются ниже.

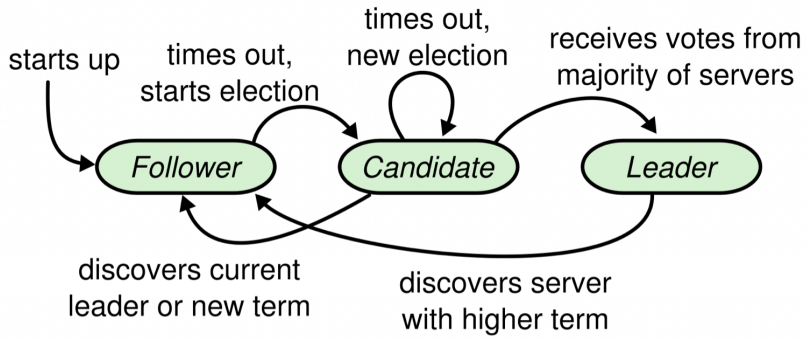


Figure 4: Server states. Followers only respond to requests from other servers. If a follower receives no communication, it becomes a candidate and initiates an election. A candidate that receives votes from a majority of the full cluster becomes the new leader. Leaders typically operate until they fail.

Raft делит время на *эпохи* произвольной длины, как показано на рисунке 5. Эпохи нумеруются последовательными целыми числами. Каждая эпоха начинается с *выборов*, на которых один или несколько кандидатов пытаются стать лидером, как описано в Разделе 5.2. Если кандидат побеждает на выборах, то он остается лидером до конца эпохи. В некоторых ситуациях выборы приведут к разделению голосов. В этом случае эпоха закончится без лидера; новая эпоха (с новыми выборами) начнется в ближайшее время. Raft гарантирует, что в данной эпохе будет не более одного лидера.

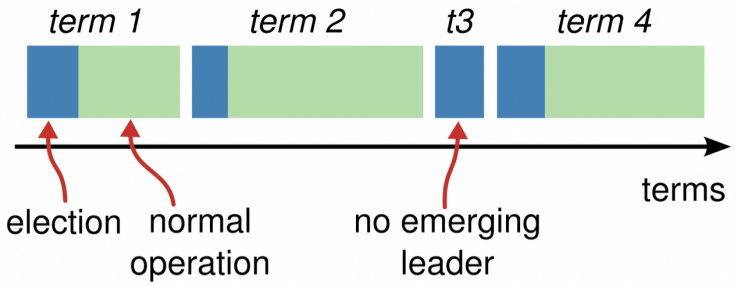


Figure 5: Time is divided into terms, and each term begins with an election. After a successful election, a single leader manages the cluster until the end of the term. Some elections fail, in which case the term ends without choosing a leader. The transitions between terms may be observed at different times on different servers.

Разные серверы могут наблюдать переходы между эпохами в разное время, а в некоторых случаях сервер может не увидеть выборы или даже целые эпохи. Эпохи играют роль логических часов Raft и позволяют серверам обнаруживать устаревшую информацию, например прежних лидеров. Каждый сервер хранит номер *текущей эпохи*, который монотонно увеличивается. Серверы обмениваются номерами эпох при каждом взаимодействии; сервер с меньшим номером обновляет его до большего значения. Если кандидат или лидер обнаруживает, что его эпоха устарела, он немедленно становится ведомым. Запрос с устаревшим номером эпохи отклоняется.

Серверы Raft взаимодействуют с помощью удаленных вызовов процедур (RPC), а базовый алгоритм консенсуса требует только двух типов RPC. RPC RequestVote инициируются кандидатами во время выборов (раздел 5.2), а RPC AppendEntries инициируются лидерами для репликации записей журнала и обеспечения формы heartbeat-сообщения (раздел 5.3). В разделе 7 добавлен третий RPC для передачи

снимков между серверами. Серверы повторяют RPC, если не получают ответа своевременно, и выдают RPC параллельно для достижения наилучшей производительности.

5.2 Выборы лидера

Raft использует механизм heartbeat-сообщений для запуска выборов лидера. Когда серверы запускаются, они начинаются как ведомые. Сервер остается в состоянии ведомого до тех пор, пока он получает действительные RPC от лидера или кандидата. Лидеры отправляют периодические heartbeat-сообщения (AppendEntries RPC, не содержащие записей журнала) всем ведомым, чтобы поддерживать свои полномочия. Если ведомый не получает сообщения в течение периода времени, называемого *таймаутом выборов*, он предполагает, что жизнеспособного лидера нет, и начинает выборы для выбора нового лидера.

Чтобы начать выборы, ведомый увеличивает номер текущей эпохи и переходит в состояние кандидата. Затем он голосует за себя и выдает RPC RequestVote параллельно каждому другому серверу в кластере. Кандидат продолжает находиться в этом состоянии до тех пор, пока не произойдет одно из трех событий:

1. оно побеждает на выборах
2. другой сервер становится лидером
3. Проходит некоторое время, а победителя нет.

Эти результаты обсуждаются отдельно в пунктах ниже.

Кандидат побеждает на выборах, если он получает голоса от большинства серверов в полном кластере за одну и ту же эпоху. Каждый сервер будет голосовать не более чем за одного кандидата в пределах одной эпохи в порядке очереди (примечание: раздел 5.4 добавляет дополнительное ограничение на голоса). Правило большинства гарантирует, что не более одного кандидата может победить на выборах на определённую эпоху (свойство Election Safety на рисунке 3). Если кандидат побеждает на выборах, он становится лидером. Затем он отправляет контрольные сообщения всем остальным серверам, чтобы установить свои полномочия и предотвратить новые выборы.

В ожидании голосов кандидат может получить AppendEntries RPC от другого сервера, претендующего на звание лидера. Если номер эпохи лидера (указанный в RPC) не меньше номера текущей эпохи кандидата, то кандидат признает лидера легитимным и возвращается в состояние ведомого. Если номер эпохи в RPC меньше номера текущей эпохи кандидата, то кандидат отклоняет RPC и продолжает оставаться в состоянии кандидата.

Третий возможный результат заключается в том, что кандидат не выигрывает и не проигрывает выборы: если многие ведомые станут кандидатами одновременно, голоса могут быть разделены так, что ни один кандидат не получит большинства. Когда это произойдет, у каждого кандидата истечет таймаут, и он начнет новые выборы, увеличив номер своей эпохи и инициировав еще один раунд RequestVote RPC. Однако без дополнительных мер раскол голосов может повторяться бесконечно.

Raft использует случайные таймауты выборов, чтобы гарантировать, что разделение голосов происходит редко и быстро разрешается. Прежде всего, чтобы предотвратить разделение голосов, таймауты выборов выбираются случайным образом из фиксированного интервала (например, 150–300 мс). Это распределяет серверы так, что в большинстве случаев таймаут будет только у одного сервера; он побеждает на выборах и отправляет heartbeat-сообщения до истечения времени ожидания других серверов. Тот же механизм используется для обработки разделения голосов. Каждый кандидат перезапускает свой рандомизированный таймаут выборов в начале выборов и ждет истечения этого таймаута, прежде чем начать следующие выборы; это снижает вероятность еще одного разделения голосов на новых выборах. В разделе 9.3 показано, что при таком подходе лидер быстро избирается.

Выборы являются примером того, как понятность определяла наш выбор между альтернативами дизайна. Изначально мы планировали использовать рейтинговую систему: каждому кандидату присваивался уникальный ранг, по которому можно было выбирать между конкурирующими кандидатами. Если кандидат обнаруживал другого кандидата с более высоким рейтингом, он возвращался в состояние ведомого, чтобы кандидату с более высоким рейтингом было легче победить на следующих выборах. Мы обнаружили, что этот подход создает тонкие проблемы с доступностью (серверу с более низким рейтингом

может потребоваться таймаут и снова стать кандидатом, если сервер с более высоким рейтингом выйдет из строя, но если это произойдет слишком рано, он может сбросить прогресс в выборе лидера). Мы несколько раз вносили корректировки в алгоритм, но после каждой корректировки появлялись новые угловые случаи. В конце концов мы пришли к выводу, что подход с рандомизированными повторами более очевиден и понятен.

5.3 Репликация журналов

После избрания лидера он начинает обслуживать запросы клиентов. Каждый клиентский запрос содержит команду, которую должны выполнить реплицированные конечные автоматы. Лидер добавляет команду в свой журнал как новую запись, затем запускает RPC AppendEntries параллельно каждому из других серверов для репликации записи. Когда запись была безопасно реплицирована (как описано ниже), лидер применяет запись к своему конечному автомату и возвращает результат этого выполнения клиенту. Если ведомые серверы выходят из строя или работают медленно, или если сетевые пакеты потеряны, лидер повторяет RPC AppendEntries бесконечно (даже после того, как он ответил клиенту), пока все ведомые в конечном итоге не сохранят все записи журнала.

Журналы организованы, как показано на рисунке 6. В каждой записи журнала хранится команда конечного автомата вместе с номером эпохи, когда запись была получена лидером. Номера эпох в записях журнала используются для обнаружения несоответствий между журналами и для обеспечения некоторых свойств, показанных на рисунке 3. Каждая запись журнала также имеет целочисленный индекс, определяющий ее положение в журнале.

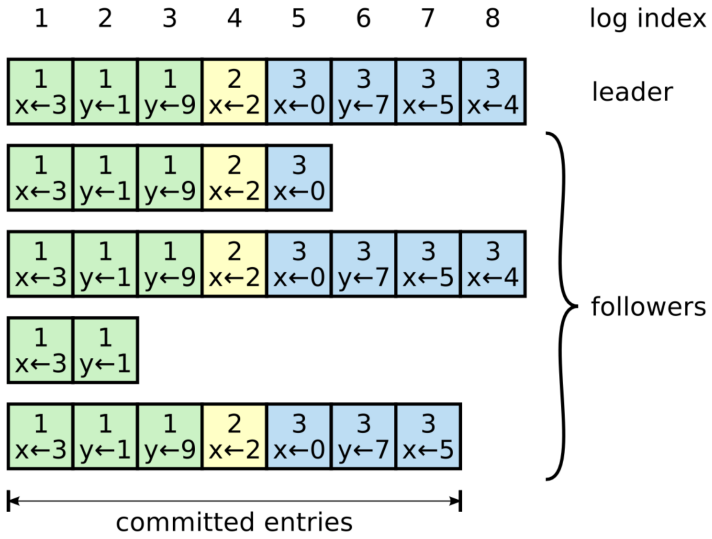


Figure 6: Logs are composed of entries, which are numbered sequentially. Each entry contains the term in which it was created (the number in each box) and a command for the state machine. An entry is considered *committed* if it is safe for that entry to be applied to state machines.

Лидер решает, когда безопасно применять запись журнала к конечным машинам; такая запись называется *зафиксированной*. Raft гарантирует, что зафиксированные записи устойчивы и в конечном итоге будут выполнены всеми доступными конечными автоматами. Запись журнала фиксируется, как только лидер, создавший запись, реплицирует ее на большинстве серверов (например, запись 7 на рис. 6). При этом также фиксируются все предыдущие записи в журнале лидера, включая записи, созданные предыдущими лидерами. В разделе 5.4 обсуждаются некоторые тонкости применения этого правила после смены лидера, а также показано, что такое определение фиксации является безопасным. Лидер отслеживает самый высокий индекс, который, как ему известно, зафиксирован, и включает этот индекс в будущие RPC AppendEntries (включая heartbeat-сообщения), чтобы другие серверы в конечном итоге узнали об этом. Как

только ведомый узнает, что запись в журнале зафиксирована, он применяет эту запись к своему локальному конечному автомату (в порядке записей журнала).

Мы разработали механизм журналов Raft для поддержания высокого уровня согласованности между журналами на разных серверах. Это не только упрощает поведение системы и делает его более предсказуемым, но и является важной составляющей обеспечения безопасности. Raft поддерживает следующие свойства, которые вместе составляют свойство Log Matching на рисунке 3:

- Если две записи в разных журналах имеют одинаковые индекс и номер эпохи, то они хранят одну и ту же команду.
- Если две записи в разных журналах имеют одинаковые индекс и номер эпохи, то журналы идентичны во всех предыдущих записях.

Первое свойство следует из того, что лидер создает не более одной записи с заданным индексом журнала за определенный период, и записи журнала никогда не меняют своего положения в журнале. Второе свойство гарантируется простой проверкой согласованности, выполняемой AppendEntries. При отправке AppendEntries RPC лидер включает в свой журнал индекс и номер эпохи записи, которая непосредственно предшествует новым записям. Если ведомый не находит в своем журнале записи с тем же индексом и эпохой, он отказывается от новых записей. Проверка согласованности действует как вводный шаг: начальное пустое состояние журналов удовлетворяет свойству Log Matching, а проверка согласованности сохраняет свойство Log Matching при каждом расширении журналов. В результате всякий раз, когда AppendEntries успешно возвращается, лидер знает, что журнал ведомого идентичен его собственному журналу благодаря новым записям.

Во время нормальной работы журналы лидера и ведомых остаются согласованными, поэтому проверка согласованности AppendEntries никогда не дает сбоя. Однако сбои лидера могут привести к тому, что журналы станут противоречивыми (старый лидер, возможно, не полностью реплицировал все записи в своем журнале). Эти несоответствия могут усугубляться серией сбоев лидеров и ведомых. На рисунке 7 показано, чем журналы ведомых могут отличаться от журналов нового лидера. У ведомого могут отсутствовать записи, присутствующие в ведущем элементе, могут быть дополнительные записи, отсутствующие в ведущем элементе, или и то, и другое. Отсутствующие и посторонние записи в журнале могут охватывать несколько эпох.

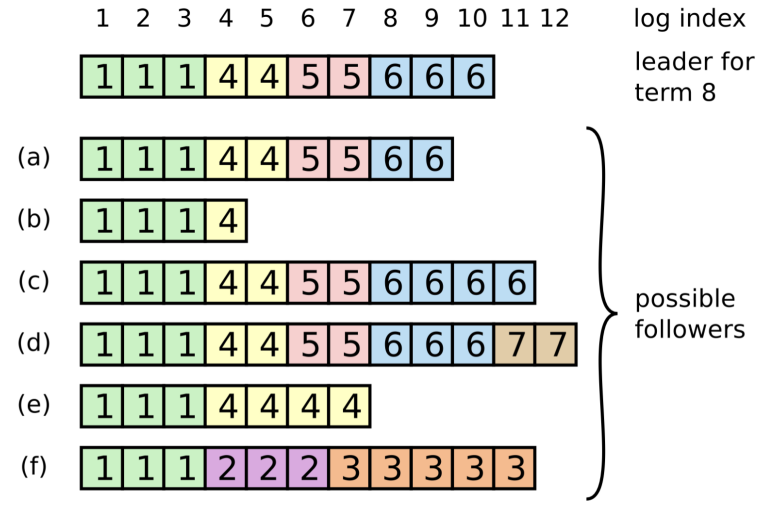


Figure 7: When the leader at the top comes to power, it is possible that any of scenarios (a–f) could occur in follower logs. Each box represents one log entry; the number in the box is its term. A follower may be missing entries (a–b), may have extra uncommitted entries (c–d), or both (e–f). For example, scenario (f) could occur if that server was the leader for term 2, added several entries to its log, then crashed before committing any of them; it restarted quickly, became leader for term 3, and added a few more entries to its log; before any of the entries in either term 2 or term 3 were committed, the server crashed again and remained down for several terms.

В Raft лидер устраняет несоответствия, заставляя журналы ведомых дублировать свои собственные. Это означает, что конфликтующие записи в журналах ведомых будут перезаписаны записями из журнала лидера. Раздел 5.4 покажет, что это безопасно в сочетании с еще одним ограничением.

Чтобы привести журнал ведомого в соответствие со своим собственным, лидер должен найти последнюю запись журнала, в которой оба журнала совпадают, удалить все записи в журнале ведомого после этой точки и отправить ведомому все записи лидера после этой точки. Все эти действия происходят в ответ на проверку согласованности, выполняемую RPC AppendEntries. Лидер сохраняет *nextIndex* для каждого ведомого, который является индексом следующей записи журнала, которую лидер отправит этому ведомому. Когда лидер впервые приходит к власти, он инициализирует все значения nextIndex индексом сразу после последнего значения в своем журнале (11 на рисунке 7). Если журнал ведомого не соответствует журналу ведущего, проверка согласованности AppendEntries завершится неудачей в следующем AppendEntries RPC. После отклонения ведущий уменьшает nextIndex и повторяет AppendEntries RPC. В конце концов nextIndex достигнет точки, в которой журналы лидера и ведомого совпадают. Когда это произойдет, AppendEntries завершится успешно, что удалит все конфликтующие записи в журнале ведомого и добавит записи из журнала лидера (если таковые имеются). Как только AppendEntries преуспеет, журнал ведомого согласуется с журналом лидера и останется таким до конца эпохи.

При желании протокол можно оптимизировать для уменьшения количества отклоненных RPC AppendEntries. Например, при отклонении запроса AppendEntries ведомый может включить номер эпохи конфликтующей записи и первый индекс, который он сохраняет для этой эпохи. Имея эту информацию, лидер может уменьшить nextIndex, чтобы обойти все конфликтующие записи в этой эпохе; один AppendEntries RPC потребуется для каждой эпохи с конфликтующими записями, а не один RPC для каждой записи. На практике мы сомневаемся в необходимости такой оптимизации, поскольку сбои случаются нечасто и маловероятно, что будет много противоречивых записей.

Благодаря этому механизму лидеру не нужно предпринимать никаких специальных действий для восстановления целостности журнала, когда он приходит к власти. Он только начинает нормальную работу, и журналы автоматически сходятся в ответ на сбои проверки согласованности AppendEntries.

Лидер никогда не перезаписывает и не удаляет записи в своем журнале (свойство Leader Append-Only на рис. 3).

Этот механизм репликации журналов демонстрирует желаемые свойства консенсуса, описанные в разделе 2: Raft может принимать, реплицировать и применять новые записи журнала, пока работает большинство серверов; в обычном случае новая запись может быть реплицирована за один раунд RPC на большую часть кластера; и один медленный ведомый не повлияет на производительность.

5.4 Безопасность

В предыдущих разделах описывалось, как Raft выбирает лидеров и реплицирует записи журнала. Однако описанных до сих пор механизмов недостаточно, чтобы гарантировать, что каждый конечный автомат выполняет одни и те же команды в одном и том же порядке. Например, ведомый может быть недоступен, пока лидер фиксирует несколько записей в журнале, тогда его можно выбрать лидером и перезаписать эти записи новыми; в результате разные конечные автоматы могут выполнять разные последовательности команд.

Этот раздел завершает алгоритм Raft, добавляя ограничение на то, какие серверы могут быть избраны лидерами. Ограничение гарантирует, что лидер для любой заданной эпохи содержит все записи, зафиксированные в предыдущих эпохах (свойство Leader Completeness на рисунке 3). Учитывая ограничения на выборы, мы затем уточняем правила фиксации. Наконец, мы представляем эскиз доказательства свойства Leader Completeness и показываем, как оно приводит к правильному поведению реплицируемого конечного автомата.

5.4.1 Ограничение выборов

В любом алгоритме консенсуса на основе лидера лидер должен в конечном итоге сохранить все зафиксированные записи журнала. В некоторых алгоритмах консенсуса, таких как Viewstamped Replication [22], лидер может быть избран, даже если он изначально не содержит всех зафиксированных записей. Эти алгоритмы содержат дополнительные механизмы для выявления недостающих записей и передачи их новому лидеру либо во время избирательного процесса, либо вскоре после него. К сожалению, это приводит к появлению значительного дополнительного механизма и сложности. Raft использует более простой подход, при котором он гарантирует, что все зафиксированные записи из предыдущих эпох присутствуют у каждого нового лидера с момента его избрания, без необходимости передавать эти записи лидеру. Это означает, что записи журнала передаются только в одном направлении, от лидеров к ведомым, и лидеры никогда не перезаписывают существующие записи в своих журналах.

Raft использует процесс голосования, чтобы предотвратить победу кандидата на выборах, если его журнал не содержит всех зафиксированных записей. Чтобы быть избранным, кандидат должен связаться с большинством кластера, а это означает, что каждая зафиксированная запись должна присутствовать хотя бы на одном из этих серверов. Если журнал кандидата по крайней мере так же актуален, как и любой другой журнал в этом большинстве (где «актуальный» определен точно ниже), то в нем будут храниться все зафиксированные записи. RequestVote RPC реализует это ограничение: RPC включает информацию о журнале кандидата, и избиратель отказывает в голосовании, если его собственный журнал более актуален, чем журнал кандидата.

Raft определяет, какой из двух журналов более актуален, путем сравнения индекса и номера эпохи последних записей в журналах. Если последние записи журналов относятся к разным эпохам, то журнал с более поздней эпохой является более актуальным. Если журналы заканчиваются одной и той же эпохой, то тот журнал, который длиннее, является более актуальным.

5.4.2 Фиксация записей из предыдущих эпох

Как описано в разделе 5.3, лидер знает, что запись своей текущей эпохи зафиксирована, как только эта запись сохраняется на большинстве серверов. Если лидер выходит из строя до фиксации записи, будущие лидеры попытаются завершить репликацию записи. Однако лидер не может сразу сделать вывод, что запись из предыдущей эпохи зафиксирована, если она хранится на большинстве серверов. Рисунок 8 иллюстрирует ситуацию, когда старая запись журнала хранится на большинстве серверов, но все же может быть перезаписана будущим лидером.

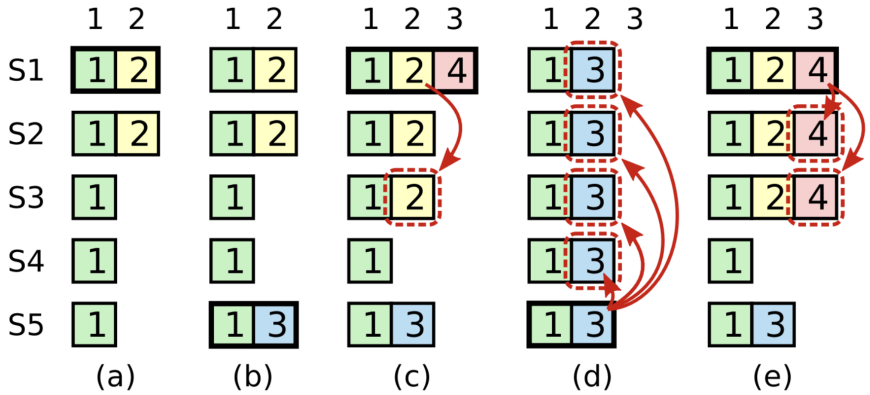


Figure 8: A time sequence showing why a leader cannot determine commitment using log entries from older terms. In (a) S1 is leader and partially replicates the log entry at index 2. In (b) S1 crashes; S5 is elected leader for term 3 with votes from S3, S4, and itself, and accepts a different entry at log index 2. In (c) S5 crashes; S1 restarts, is elected leader, and continues replication. At this point, the log entry from term 2 has been replicated on a majority of the servers, but it is not committed. If S1 crashes as in (d), S5 could be elected leader (with votes from S2, S3, and S4) and overwrite the entry with its own entry from term 3. However, if S1 replicates an entry from its current term on a majority of the servers before crashing, as in (e), then this entry is committed (S5 cannot win an election). At this point all preceding entries in the log are committed as well.

Чтобы избежать проблем, подобных той, что показана на рисунке 8, Raft никогда не фиксирует записи журнала из предыдущих эпох путем подсчета реплик. Путем подсчета реплик фиксируются только записи журнала текущей эпохи лидера; как только запись из текущей эпохи была зафиксирована таким образом, все предыдущие записи фиксируются косвенно из-за свойства Log Matching. В некоторых ситуациях лидер может с уверенностью заключить, что зафиксирована более старая запись журнала (например, если эта запись хранится на каждом сервере), но Raft для простоты использует более консервативный подход.

Raft создает дополнительную сложность в правилах фиксации, поскольку записи журнала сохраняют исходные номера эпох, когда лидер копирует записи из предыдущих эпох. В других алгоритмах консенсуса, если новый лидер повторяет записи из предыдущих «эпох», он должен делать это со своим новым «номером эпохи». Подход Raft упрощает анализ записей журнала, поскольку они сохраняют один и тот же номер эпохи с течением времени и в разных журналах. Кроме того, новые лидеры в Raft отправляют меньше записей журнала из предыдущих эпох, чем в других алгоритмах (другие алгоритмы должны отправлять избыточные записи журнала, чтобы перенумеровать их, прежде чем их можно будет зафиксировать).

5.4.3 Обоснование безопасности

Имея полный алгоритм Raft, мы теперь можем более точно утверждать, что свойство Leader Completeness справедливо (этот аргумент основан на доказательстве безопасности; см. раздел 9.2). Предположим, что свойство Leader Completeness не выполнено, затем докажем противоречие. Предположим, лидер эпохи T (leaderT) фиксирует запись в журнале из своей эпохи, но эта запись журнала не сохраняется лидером какой-либо будущей эпохи. Рассмотрим наименьшую эпоху U > T, лидер которой (leaderU) не сохраняет запись.

1. Зафиксированная запись должна отсутствовать в журнале leaderU на момент ее выбора (лидеры никогда не удаляют и не перезаписывают записи).
2. leaderT реплицировал запись на большую часть кластера, а leaderU получил голоса от большинства кластера. Таким образом, по крайней мере один сервер («избиратель») принял запись от лидера T и проголосовал за лидера U, как показано на рисунке 9. Избиратель является ключом к достижению противоречия.
3. Избиратель должен принять зафиксированную запись от leaderT *до* голосования за leaderU; в противном случае он бы отклонил запрос AppendEntries от лидера T (номер его текущей эпохи был бы больше, чем T).
4. Избиратель по-прежнему сохранял запись, когда голосовал за leaderU, поскольку каждый промежуточный лидер содержал запись (по предположению), лидеры никогда не удаляют записи, а ведомые удаляют записи только в том случае, если они конфликтуют с лидером.
5. Избиратель отдал свой голос leaderU, поэтому журнал leaderU должен быть таким же актуальным, как и журнал избирателя. Это приводит к одному из двух противоречий.
6. Во-первых, если избиратель и лидер U использовали один и тот же номер эпохи последней записи, то длина журнала leaderU должна быть не меньше длины журнала избирателя, поэтому его журнал содержал все записи в журнале избирателя. Это противоречие, поскольку избиратель содержал зафиксированную запись, а leaderU предполагалось, что ее нет.
7. В противном случае последний член журнала leaderU должен был быть больше, чем у избирателя. Более того, оно было больше, чем T, поскольку номер эпохи последней записи в журнале избирателя был как минимум T (он содержит зафиксированную запись из эпохи T). Предыдущий лидер, создавший последнюю запись журнала leaderU, должен был содержать зафиксированную запись в своем журнале (по предположению). Тогда, согласно свойству Log Matching, журнал leaderU также должен содержать зафиксированную запись, что является противоречием.
8. Это завершает противоречие. Таким образом, лидеры всех эпох, больших, чем T, должны содержать все записи из терма T, зафиксированные в терме T.
9. Свойство Log Matching гарантирует, что будущие лидеры также будут содержать записи, зафиксированные косвенно, например индекс 2 на рисунке 8(d).

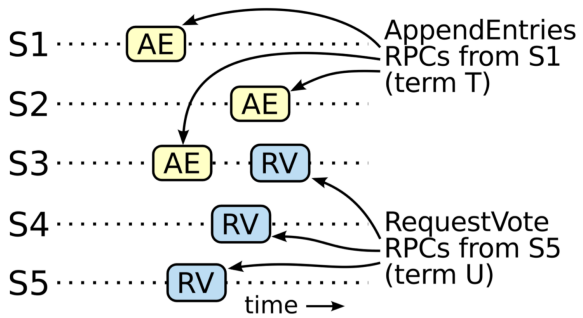


Figure 9: If S1 (leader for term T) commits a new log entry from its term, and S5 is elected leader for a later term U, then there must be at least one server (S3) that accepted the log entry and also voted for S5.

Учитывая свойство Leader Completeness, мы можем доказать свойство State Machine Safety на рисунке 3, где говорится, что если сервер применил запись журнала по заданному индексу к своему конечному автомату, ни один другой сервер никогда не применит другую запись журнала для того же индекса. В то время, когда сервер применяет запись журнала к своему конечному автомату, его журнал должен быть идентичен журналу лидера, вплоть до этой записи, и запись должна быть зафиксирована. Теперь рассмотрим самую раннюю эпоху, в которой любой сервер применяет заданный индекс журнала; Свойство Log Completeness гарантирует, что лидеры для всех более высоких эпох будут хранить одну и ту же запись журнала, поэтому серверы, применяющие индекс в более поздних эпохах, будут применять одно и то же значение. Таким образом, свойство State Machine Safety сохраняется.

Наконец, Raft требует, чтобы серверы применяли записи в порядке индекса журнала. В сочетании со свойством State Machine Safety это означает, что все серверы будут применять один и тот же набор записей журнала к своим конечным автоматам в одном и том же порядке.

5.5 Сбои ведомых и кандидатов

До этого момента мы фокусировались на неудачах лидеров. Сбои ведомого и кандидата гораздо проще обрабатывать, чем сбои лидера, и оба они обрабатываются одинаково. Если ведомый или кандидат выйдет из строя, то отправленные ему будущие RPC RequestVote и AppendEntries завершатся неудачей. Raft обрабатывает эти ошибки, повторяя попытки бесконечно; если вышедший из строя сервер перезапустится, RPC завершится успешно. Если сервер выйдет из строя после завершения RPC, но до ответа, то после перезапуска он снова получит тот же RPC. RPC Raft идемпотентны, поэтому это не причиняет вреда. Например, если ведомый получает запрос AppendEntries, который включает записи журнала, уже присутствующие в его журнале, он игнорирует эти записи в новом запросе.

5.6 Временные параметры и доступность

Одно из наших требований к Raft заключается в том, что безопасность не должна зависеть от времени: система не должна выдавать неправильные результаты только потому, что какое-то событие происходит быстрее или медленнее, чем ожидалось. Однако доступность (способность системы своевременно реагировать на запросы клиентов) неизбежно должна зависеть от времени. Например, если обмен сообщениями длится дольше среднего интервала между сбоями серверов, кандидаты не успеют победить на выборах; без устойчивого лидера Raft не сможет добиться прогресса.

Выборы лидера — это аспект Raft, где время имеет решающее значение. Raft сможет выбирать и поддерживать постоянного лидера, пока система удовлетворяет следующим *требованиям к времени*:

$$\textbf{broadcastTime} \ll \textbf{electionTimeout} \ll \textbf{MTBF}$$

В этом неравенстве *broadcastTime* — это среднее время, необходимое серверу для отправки RPC параллельно каждому серверу в кластере и получения от них ответов; *electionTimeout* — таймаут выборов, описанный в разделе 5.2; и *MTBF* — среднее время между сбоями для одного сервера. Время трансляции должно быть на порядок меньше таймаута выборов, чтобы лидеры могли надежно отправлять heartbeat-сообщения, необходимые для того, чтобы удержать ведомых от начала выборов; учитывая рандомизированный подход, используемый для таймаутов выборов, это неравенство также делает маловероятным разделение голосов. Таймаут выборов должен быть на несколько порядков меньше, чем MTBF, чтобы система стабильно развивалась. Когда лидер выйдет из строя, система будет недоступна примерно в течение времени ожидания выборов; нам бы хотелось, чтобы это занимало лишь небольшую часть общего времени.

Время трансляции и MTBF являются свойствами базовой системы, а время ожидания выборов — это то, что мы должны выбрать. RPC Raft обычно требуют, чтобы получатель сохранял информацию в стабильном хранилище, поэтому время трансляции может варьироваться от 0,5 до 20 мс, в зависимости от технологии хранения. В результате таймаут выборов, вероятно, будет где-то между 10 и 500 мс. Типичное время безотказной работы сервера составляет несколько месяцев и более, что легко удовлетворяет требованиям по времени.

6 Изменения членства в кластере

До сих пор мы предполагали, что *конфигурация* кластера (набор серверов, участвующих в алгоритме консенсуса) фиксирована. На практике иногда возникает необходимость изменить конфигурацию, например, для замены серверов при их выходе из строя или изменения степени репликации. Хотя это можно сделать, отключив весь кластер, обновив файлы конфигурации и затем перезапустив кластер, это оставит кластер недоступным во время переключения. Кроме того, если выполняются какие-либо действия вручную, существует риск ошибки оператора. Чтобы избежать этих проблем, мы решили автоматизировать изменения конфигурации и включить их в алгоритм консенсуса Raft.

Чтобы механизм смены конфигурации был безопасным, во время перехода не должно быть такого момента, когда два лидера могли бы быть избраны на одну и ту же эпоху. К сожалению, любой подход, при котором серверы напрямую переключаются со старой конфигурации на новую, небезопасен. Невозможно атомарно переключить все серверы одновременно, поэтому во время перехода кластер потенциально может разделиться на два независимых большинства (см. рис. 10).

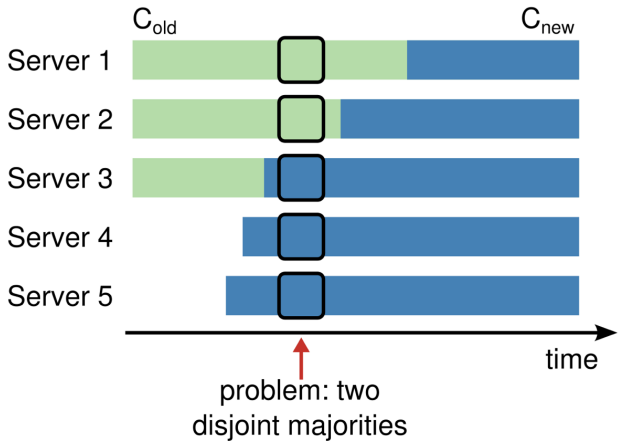


Figure 10: Switching directly from one configuration to another is unsafe because different servers will switch at different times. In this example, the cluster grows from three servers to five. Unfortunately, there is a point in time where two different leaders can be elected for the same term, one with a majority of the old configuration (C_{old}) and another with a majority of the new configuration (C_{new}).

В целях обеспечения безопасности при изменении конфигурации необходимо использовать двухэтапный подход. Существуют различные способы реализации этих двух этапов. Например, некоторые системы (например, [22]) используют первую фазу для отключения старой конфигурации, чтобы она не могла обрабатывать запросы клиентов; затем второй этап включает новую конфигурацию. В Raft кластер сначала переключается на переходную конфигурацию, которую мы называем *совместным консенсусом*; как только совместный консенсус достигнут, система переходит в новую конфигурацию. Совместный консенсус сочетает в себе как старую, так и новую конфигурации:

- Записи журнала реплицируются на все серверы в обеих конфигурациях.
- Лидером может быть любой сервер любой конфигурации.
- Соглашение (для выборов и фиксации записей) требует отдельного большинства от *как* старой, так и новой конфигурации.

Совместный консенсус позволяет отдельным серверам переключаться между конфигурациями в разное время без ущерба для безопасности. Более того, совместный консенсус позволяет кластеру продолжать обслуживать запросы клиентов во время изменения конфигурации.

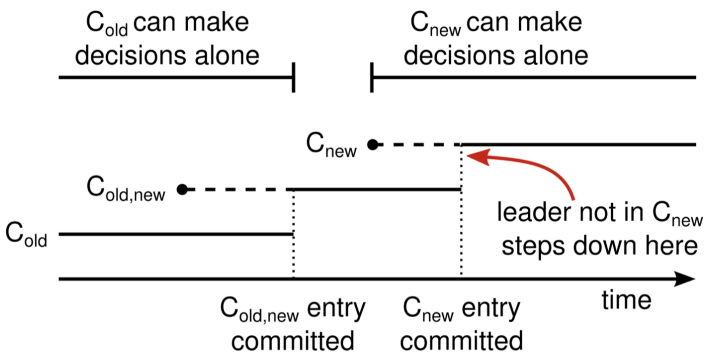


Figure 11: Timeline for a configuration change. Dashed lines show configuration entries that have been created but not committed, and solid lines show the latest committed configuration entry. The leader first creates the $C_{old,new}$ configuration entry in its log and commits it to $C_{old,new}$ (a majority of C_{old} and a majority of C_{new}). Then it creates the C_{new} entry and commits it to a majority of C_{new}. There is no point in time in which C_{old} and C_{new} can both make decisions independently.

Конфигурации кластера хранятся и передаются с помощью специальных записей в реплицируемом журнале; На рис. 11 показан процесс изменения конфигурации. Когда лидер получает запрос на изменение конфигурации с C_{old} на C_{new}, он сохраняет конфигурацию для совместного консенсуса (C_{old, new} на рисунке) в виде записи журнала и реплицирует эту запись, используя механизмы, описанные ранее. Как только данный сервер добавляет новую запись конфигурации в свой журнал, он использует эту конфигурацию для всех будущих решений (сервер всегда использует последнюю конфигурацию в своем журнале, независимо от того, зафиксирована ли запись). Это означает, что лидер будет использовать правила C_{old},new, чтобы определить, когда будет зафиксирована запись журнала для C_{old},new. В случае сбоя лидера новый лидер может быть выбран под C_{old} или C_{old, new}, в зависимости от того, получил ли победивший кандидат C_{old, new}. В любом случае C_{new} не может принимать односторонние решения в течение этого периода.

После фиксации C_{old, new} ни C_{old}, ни C_{new} не могут принимать решения без одобрения друг друга, а свойство Leader Completeness гарантирует, что лидером могут быть выбраны только серверы с записью журнала C_{old,new}. Теперь лидер может безопасно создать запись в журнале, описывающую C_{new}, и реплицировать ее в кластер. Опять же, эта конфигурация вступит в силу на каждом сервере, как только она будет обнаружена. Когда новая конфигурация зафиксирована в соответствии с правилами C_{new}, старая конфигурация не имеет значения, и серверы, не входящие в новую конфигурацию, могут быть отключены. Как показано на рисунке 11, не бывает времени, когда C_{old} и C_{new} могут принимать односторонние решения; это гарантирует безопасность.

При реконфигурации необходимо решить еще три проблемы. Первая проблема заключается в том, что новые серверы изначально могут не хранить записи журнала. Если они будут добавлены в кластер в этом состоянии, им может потребоваться некоторое время, чтобы наверстать упущенное, и в течение этого времени может оказаться невозможным фиксировать новые записи журнала. Чтобы избежать пробелов в доступности, Raft вводит дополнительную фазу перед изменением конфигурации, на которой новые серверы присоединяются к кластеру в качестве членов без права голоса (лидер реплицирует для них записи журнала, но они не учитываются для большинства). Как только новые серверы догонят остальную часть кластера, реконфигурацию можно будет продолжить, как описано выше.

Вторая проблема заключается в том, что лидер кластера может не входить в новую конфигурацию. В этом случае лидер уходит (возвращается в состояние ведомого) после того, как он зафиксировал запись в журнале C_{new}. Это означает, что будет период времени (во время фиксации C_{new}), когда лидер будет управлять кластером, который не включает себя; он реплицирует записи журнала, но не учитывает себя в большинстве. Переход лидера происходит при фиксации C_{new}, поскольку это первая точка, когда новая конфигурация может работать независимо (всегда можно будет выбрать лидера из C_{new}). До

этого момента лидером может быть избран только сервер C_{old}.

Третья проблема заключается в том, что удаленные серверы (не находящиеся в C_{new}) могут нарушить работу кластера. Эти серверы не будут получать heartbeat-сообщения, поэтому они дождутся истечения таймаута и начнут новые выборы. Затем они отправят RPC RequestVote с новыми номерами эпох, и это приведет к тому, что текущий лидер вернется в состояние ведомого. В конечном итоге будет избран новый лидер, но время ожидания удаленных серверов снова истечет, и процесс повторится, что приведет к ухудшению доступности.

Чтобы предотвратить эту проблему, серверы игнорируют RPC RequestVote, когда считают, что текущий лидер существует. В частности, если сервер получает RequestVote RPC в течение минимального времени ожидания выборов от текущего лидера, он не обновляет номер своей эпохи и не предоставляет свой голос. Это не влияет на обычные выборы, когда каждый сервер ожидает как минимум минимального таймаута выборов, прежде чем начать выборы. Однако это помогает избежать сбоев из-за удаленных серверов: если лидер может доставлять heartbeat-сообщения серверам кластера, то он не будет свергнут из-за большего числа эпох.

7 Компактация журнала

Журнал Raft увеличивается во время нормальной работы и включает в себя больше клиентских запросов, но в практической системе он не может расти без ограничений. По мере того как журнал становится длиннее, он занимает больше места и требует больше времени для воспроизведения. В конечном итоге это приведет к проблемам с доступностью без какого-либо механизма для удаления устаревшей информации, накопившейся в журнале.

Снимки — это самый простой подход к уплотнению. При создании снимков все текущее состояние системы записывается в *снимок* в стабильном хранилище, затем весь журнал до этого момента удаляется. Создание снимков используется в Chubby и ZooKeeper, а оставшаяся часть этого раздела описывает создание снимков в Raft.

Также возможны поэтапные подходы к уплотнению, такие как очистка журналов и деревья слияния с лог-структурой. Они обрабатывают часть данных одновременно, поэтому распределяют нагрузку по сжатию более равномерно во времени. Сначала они выбирают область данных, в которой накопилось множество удаленных и перезаписанных объектов, затем перезаписывают живые объекты из этой области более компактно и освобождают эту область. Это требует значительного дополнительного механизма и сложности по сравнению с созданием моментальных снимков, которое упрощает задачу, всегда работая со всем набором данных. Хотя очистка журналов потребует внесения изменений в Raft, конечные автоматы могут реализовывать деревья LSM, используя тот же интерфейс, что и создание снимков.

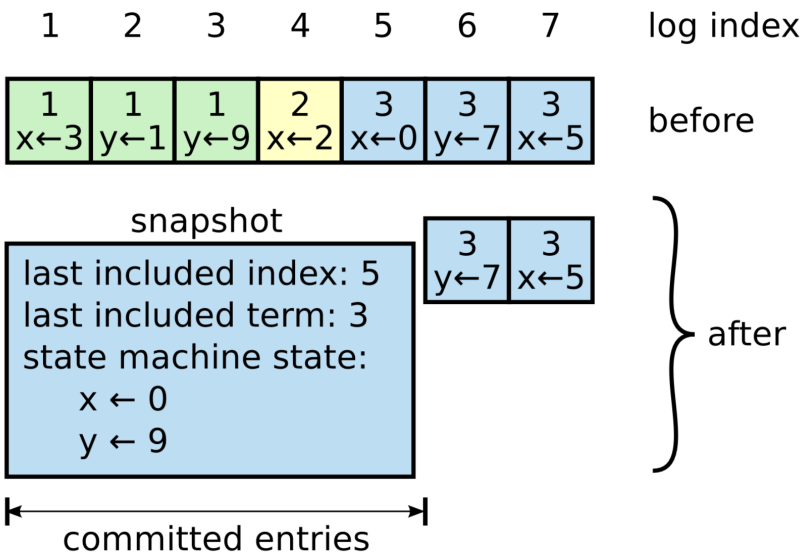
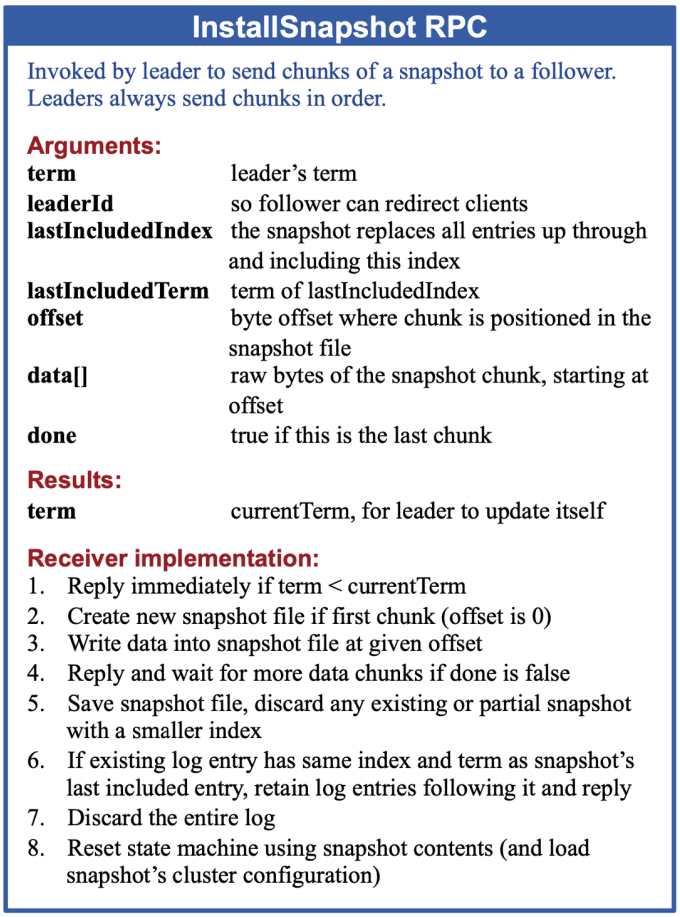


Figure 12: A server replaces the committed entries in its log (indexes 1 through 5) with a new snapshot, which stores just the current state (variables *x* and *y* in this example). The snapshot's last included index and term serve to position the snapshot in the log preceding entry 6.

На рисунке 12 показана основная идея создания снимков в Raft. Каждый сервер делает снимки независимо, охватывая только зафиксированные записи в своем журнале. Большая часть работы состоит в том, что конечный автомат записывает свое текущее состояние в снимок. Raft также включает в снимок небольшой объем метаданных: *последний включенный индекс* — это индекс последней записи в журнале, которую заменяет снимок (последняя запись, которую применил конечный автомат), а *последняя включённая эпоха* — это номер эпохи данной записи. Они сохраняются для поддержки проверки согласованности AppendEntries для первой записи журнала, следующей за моментальным снимком, поскольку для этой записи требуется индекс и номер эпохи предыдущей записи. Чтобы включить изменение членства в кластере (раздел 6), снимок также включает в журнал последнюю конфигурацию по состоянию на последний включенный индекс. Как только сервер завершит запись снимка, он может удалить все записи журнала вплоть до последнего включенного индекса, а также любой предыдущий снимок.

Хотя серверы обычно делают снимки независимо, лидеру приходится время от времени отправлять снимки отстающим ведомым. Это происходит, когда лидер уже отбросил следующую запись журнала, которую ему необходимо отправить ведомому. К счастью, при нормальной работе такая ситуация маловероятна: ведомый, отставший от лидера, уже будет иметь эту запись. Однако исключительно медленный ведомый или новый сервер, присоединяющийся к кластеру (раздел 6), этого не сделает. Чтобы обновить такого ведомого, лидер может отправить ему снимок по сети.



InstallSnapshot RPC

Invoked by leader to send chunks of a snapshot to a follower. Leaders always send chunks in order.

Arguments:

term	leader's term
leaderId	so follower can redirect clients
lastIncludedIndex	the snapshot replaces all entries up through and including this index
lastIncludedTerm	term of lastIncludedIndex
offset	byte offset where chunk is positioned in the snapshot file
data[]	raw bytes of the snapshot chunk, starting at offset
done	true if this is the last chunk

Results:

term	currentTerm, for leader to update itself

Receiver implementation:

1. Reply immediately if term < currentTerm
2. Create new snapshot file if first chunk (offset is 0)
3. Write data into snapshot file at given offset
4. Reply and wait for more data chunks if done is false
5. Save snapshot file, discard any existing or partial snapshot with a smaller index
6. If existing log entry has same index and term as snapshot's last included entry, retain log entries following it and reply
7. Discard the entire log
8. Reset state machine using snapshot contents (and load snapshot's cluster configuration)

Figure 13: A summary of the InstallSnapshot RPC. Snapshots are split into chunks for transmission; this gives the follower a sign of life with each chunk, so it can reset its election timer.

Лидер использует новый RPC под названием InstallSnapshot для отправки снимков ведомым, которые слишком сильно отстают; см. рисунок 13. Когда ведомый получает снимок с этим RPC, он должен решить, что делать с существующими записями журнала. Обычно снимок будет содержать новую информацию, которой еще нет в журнале получателя. В этом случае ведомый удаляет весь свой журнал; он целиком заменяется снимком и могут содержать незафиксированные записи, конфликтующие со снимком. Если вместо этого ведомый получает снимок, описывающий префикс его журнала (из-за повторной передачи или по ошибке), то записи журнала, охватываемые снимком, удаляются, но записи, следующие за снимком, все еще действительны и должны быть сохранены.

Этот подход к созданию снимков отходит от принципа сильного лидера Raft, поскольку ведомые могут делать снимки без ведома лидера. Однако мы считаем этот отход оправданным. Хотя наличие лидера помогает избежать противоречивых решений при достижении консенсуса, консенсус уже достигнут при создании моментальных снимков, поэтому никакие решения не противоречат друг другу. Данные по-прежнему передаются только от лидеров к ведомым, теперь только ведомые могут реорганизовать свои данные.

Мы рассмотрели альтернативный подход, основанный на лидерах, при котором только лидер создает снимок, а затем отправляет этот снимок каждому из своих ведомых. Однако это имеет два недостатка. Во-первых, отправка снимка каждому ведомому приведет к потере пропускной способности сети и замедлит процесс создания снимка. У каждого ведомого уже есть информация, необходимая для создания собственных снимков, и для сервера обычно гораздо дешевле создать снимок из своего локального состояния, чем отправлять и получать его по сети. Во-вторых, реализация лидера будет более сложной. Например, лидеру необходимо будет отправлять снимки ведомым параллельно с репликацией им новых записей журнала, чтобы не блокировать новые клиентские запросы.

Есть еще две проблемы, влияющие на производительность моментальных снимков. Во-первых, серверы должны решить, когда делать снимок. Если сервер делает снимки слишком часто, он тратит впустую полосу пропускания и энергию диска; если снимки создаются слишком редко, это рискует исчерпать емкость хранилища и увеличивает время, необходимое для воспроизведения журнала во время перезапусков. Одна из простых стратегий — сделать снимок, когда журнал достигнет фиксированного размера в байтах. Если этот размер установлен значительно больше ожидаемого размера моментального снимка, то нагрузка на полосу пропускания диска для создания моментального снимка будет небольшой.

Вторая проблема с производительностью заключается в том, что написание моментального снимка может занять значительное время, и мы не хотим, чтобы это задерживало нормальные операции. Решение состоит в том, чтобы использовать методы копирования при записи, чтобы новые обновления можно было принимать, не влияя на записываемый снимок. Например, это естественным образом поддерживают конечные автоматы, построенные на основе функциональных структур данных. Альтернативно, поддержка копирования при записи в операционной системе (например, fork в Linux) может использоваться для создания в памяти моментального снимка всего конечного автомата (наша реализация использует этот подход).

8 Взаимодействие с клиентами

В этом разделе описывается, как клиенты взаимодействуют с Raft, в том числе как клиенты находят лидера кластера и как Raft поддерживает линеаризуемую семантику. Эти проблемы применимы ко всем системам, основанным на консенсусе, а решения Raft аналогичны решениям других систем.

Клиенты Raft отправляют все свои запросы лидеру. Когда клиент впервые запускается, он подключается к случайно выбранному серверу. Если первым выбором клиента не является лидер, этот сервер отклонит запрос клиента и предоставит информацию о последнем лидере, от которого он получил информацию (запросы AppendEntries включают сетевой адрес лидера). В случае сбоя лидера время ожидания клиентских запросов истечет; затем клиенты повторяют попытку со случайно выбранными серверами.

Наша цель для Raft — реализовать линеаризуемую семантику (каждая операция выполняется мгновенно, ровно один раз, в какой-то момент между ее вызовом и ответом). Однако, как описано до сих пор, Raft может выполнять команду несколько раз: например, если лидер выходит из строя после фиксации записи в журнале, но до ответа клиенту, клиент повторит команду с новым лидером, что приведет к ее выполнению во второй раз. Решение состоит в том, чтобы клиенты присваивали каждой команде уникальные серийные номера. Затем конечный автомат отслеживает последний обработанный серийный номер для каждого клиента вместе с соответствующим ответом. Если он получает команду, серийный номер которой уже был выполнен, он отвечает немедленно, без повторного выполнения запроса.

Операции только для чтения могут обрабатываться без записи чего-либо в журнал. Однако без каких-либо дополнительных мер это может привести к риску возврата устаревших данных, поскольку лидер, ответивший на запрос, мог быть заменен новым лидером, о котором он не знает. Линеаризуемые операции чтения не должны возвращать устаревшие данные, и Raft требует двух дополнительных мер предосторожности, чтобы гарантировать это без использования журнала. Во-первых, лидер должен иметь самую свежую информацию о том, какие записи зафиксированы. Свойство Leader Completeness гарантирует, что у лидера есть все зафиксированные записи, но в начале своей эпохи он может не знать, какие именно. Чтобы это выяснить, ему необходимо зафиксировать запись из своей эпохи. Raft решает эту проблему, заставляя каждого лидера вносить в журнал пустую запись *no-op* в начале своей эпохи. Во-вторых, лидер должен проверить, был ли он свергнут, прежде чем обрабатывать запрос только для чтения (его информация может быть устаревшей, если был избран более поздний лидер). Raft решает эту проблему, заставляя лидера обмениваться heartbeat-сообщениями с большей частью кластера, прежде чем отвечать на запросы только для чтения. В качестве альтернативы лидер может положиться на механизм heartbeat-сообщений, чтобы обеспечить форму аренды, но в целях безопасности это будет зависеть от времени (предполагается ограниченный дрейф часов).

9. Реализация и оценка

Мы реализовали Raft как часть реплицируемого конечного автомата, который хранит информацию о конфигурации для RAMCloud [33] и помогает в аварийном переключении координатора RAMCloud. Реализация Raft содержит примерно 2000 строк кода C++, не считая тестов, комментариев и пустых строк. Исходный код находится в свободном доступе [23]. Существует также около 25 независимых сторонних реализаций [34] с открытым исходным кодом Raft на различных стадиях разработки, основанных на черновиках этой статьи. Также различные компании внедряют системы Raft на базе [34].

В оставшейся части этого раздела Raft оценивается по трем критериям: понятность, правильность и производительность.

9.1 Понятность

Чтобы измерить понятность Raft по сравнению с Paxos, мы провели экспериментальное исследование с участием старшекурсников и аспирантов, изучавших курс «Продвинутые операционные системы» в Стэнфордском университете и курса «Распределенные вычисления» в Калифорнийском университете в Беркли. Мы записали видеолекцию Raft и еще одну Paxos и создали соответствующие тесты. Лекция Raft охватывала содержание этой статьи, за исключением компактации журнала; лекция Paxos содержала достаточно материала для создания эквивалентного реплицируемого конечного автомата, включая одноразовый Paxos, Multi-Paxos, реконфигурацию и несколько оптимизаций, необходимых на практике (например, выборы лидера). Викторины проверяли базовое понимание алгоритмов, а также требовали от учащихся рассуждений о пограничных случаях. Каждый студент посмотрел одно видео, выполнил соответствующий тест, посмотрел второе видео и прошел второй тест. Около половины участников сначала выполнили часть Paxos, а другая половина сначала выполнила часть Raft, чтобы учесть как индивидуальные различия в производительности, так и опыт, полученный в первой части исследования. Мы сравнили баллы участников по каждому тесту, чтобы определить, продемонстрировали ли участники лучшее понимание Raft.

Мы постарались сделать сравнение Paxos и Raft максимально честным. Эксперимент отдал предпочтение Paxos по двум причинам: 15 из 43 участников сообщили, что имели некоторый опыт работы с Paxos, а видео Paxos на 14% длиннее, чем видео Raft. Как показано в Таблице 1, мы предприняли шаги по смягчению потенциальных источников систематической ошибки. Все наши материалы доступны для ознакомления [28, 31].

Источник риска	*Шаги, предпринятые для смягчения предвзятости*	*Материалы для обзора [28, 31]*
Равное качество лекций	*Один и тот же лектор для обоих. Лекция Paxos основана на существующих материалах, используемых в нескольких университетах, и улучшена из них. Лекция Paxos стала на 14% длиннее.*	*видео*
Одинаковая сложность тесты	*Вопросы сгруппированы по сложности и объединены в пары по экзаменам.*	*тесты*
Справедливая оценка	*Использовались единые критерии оценки. Оценки выставляются в случайном порядке, чередуя тесты.*	*рубрика*

Таблица 1: Опасения по поводу возможной предвзятости в отношении Paxos в исследовании, шаги, предпринятые для противодействия каждому из них, и доступные дополнительные материалы.

В среднем участники набрали на 4,9 балла больше в тесте Raft, чем в тесте Paxos (из возможных 60 баллов средний балл Raft составил 25,7, а средний балл Paxos — 20,8); На рисунке 14 показаны их индивидуальные баллы. Парный тест t показывает, что с достоверностью 95% истинное распределение оценок Raft в среднем как минимум на 2,5 балла превышает истинное распределение оценок Paxos.

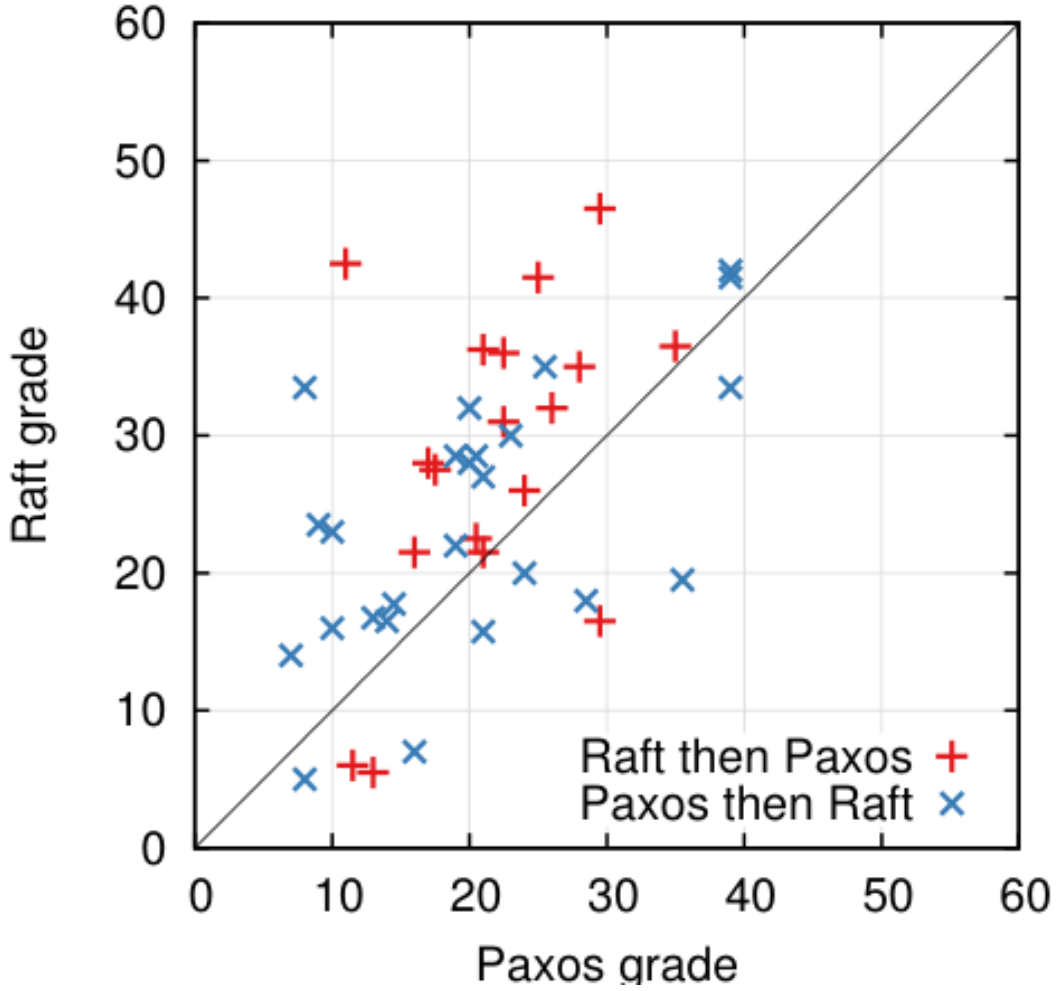


Рисунок 14: Диаграмма разброса, сравнивающая результаты 43 участников по тестам Raft и Paxos. Точки над диагональю (33) обозначают участников, набравших более высокий балл по Raft.

Мы также создали модель линейной регрессии, которая прогнозирует результаты теста новых учащихся на основе трех факторов: какой тест они прошли, степень их предыдущего опыта Paxos и порядок, в котором они изучали алгоритмы. Модель предсказывает, что выбор теста приведет к разнице в 12,5 баллов в пользу Raft. Это значительно выше наблюдаемой разницы в 4,9 балла, поскольку многие из реальных студентов уже имели опыт работы с Paxos, что значительно помогло Paxos, тогда как Raft помогло немного меньше. Любопытно, что модель также прогнозирует снижение баллов по Raft на 6,3 балла для людей, которые уже прошли тест Paxos; хотя мы не знаем почему, это кажется статистически значимым.

Мы также опросили участников после прохождения викторин, чтобы выяснить, какой алгоритм, по их мнению, будет легче реализовать или объяснить; эти результаты показаны на рисунке 15. Подавляющее большинство участников сообщили, что Raft будет легче реализовать и объяснить (33 из 41 на каждый вопрос). Однако эти ощущения, о которых сообщают сами участники, могут быть менее надежными, чем результаты тестов участников, и участники могли быть предвзяты из-за знания нашей гипотезы о том, что Raft легче понять.

Подробное обсуждение исследования с участием пользователей Raft доступно по адресу [31].

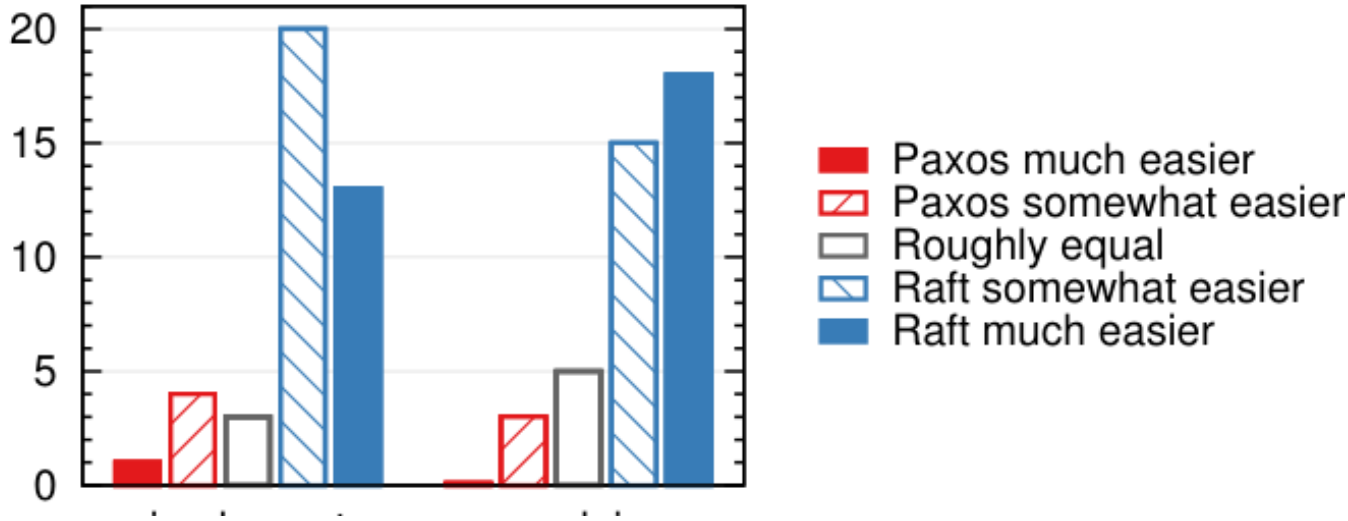


Рисунок 15: Используя 5-балльную шкалу, участников спросили (слева), какой алгоритм, по их мнению, было бы легче реализовать в функционирующей, правильной и эффективной системе, и (справа), какой из них было бы легче объяснить аспиранту CS.

9.2 Корректность

Мы разработали формальную спецификацию и доказательство безопасности механизма консенсуса, описанного в разделе 5. Формальная спецификация [31] делает информацию, представленную на рисунке 2, полностью точной с использованием языка спецификации TLA+ [17]. Он занимает около 400 строк и служит предметом доказательства. Он также полезен сам по себе для тех, кто реализует Raft. Мы механически доказали свойство полноты журнала с помощью системы проверки TLA [7]. Однако это доказательство опирается на инварианты, которые не были проверены механически (например, мы не доказали типобезопасность спецификации). Кроме того, мы написали неформальное доказательство [31] свойства State Machine Safety, которое является полным (оно основано только на спецификации) и относительно точным (объемом около 3500 слов).

9.3 Производительность

Производительность Raft аналогична производительности других алгоритмов консенсуса, таких как Paxos. Наиболее важным случаем для производительности является ситуация, когда действующий лидер реплицирует новые записи журнала. Raft достигает этого, используя минимальное количество сообщений (один обмен запрос-ответ между лидером и половиной кластера). Также возможно дальнейшее улучшение производительности Raft. Например, он легко поддерживает пакетную обработку и конвейерную обработку запросов, обеспечивая более высокую пропускную способность и меньшую задержку. В литературе были предложены различные оптимизации для других алгоритмов; многие из них можно применить к Raft, но мы оставим это для будущих разработок.

Мы использовали нашу реализацию Raft, чтобы измерить производительность алгоритма выбора лидера Raft и ответить на два вопроса. Во-первых, быстро ли сходится процесс выборов? Во-вторых, какое минимальное время простоя может быть достигнуто после сбоя лидера?

Чтобы измерить выбор лидера, мы неоднократно приводили к сбою лидера кластера из пяти серверов и рассчитывали время, необходимое для обнаружения сбоя и выбора нового лидера (см. рис. 16). В худшем случае серверы в каждом испытании имели разную длину журналов, поэтому некоторые кандидаты не имели права стать лидерами. Более того, чтобы стимулировать разделение голосов, наш тестовый сценарий запускал синхронизированную одновременную рассылку heartbeat-RPC от лидера перед завершением процесса (это примерно соответствует поведению лидера, реплицирующего новую запись журнала перед сбоем). Лидер аварийно завершал работу равномерно и случайным образом в пределах

интервала heartbeat-сообщений, который составлял половину минимального таймаута выборов для всех тестов. Таким образом, минимально возможное время простоя составило примерно половину минимального таймаута выборов.

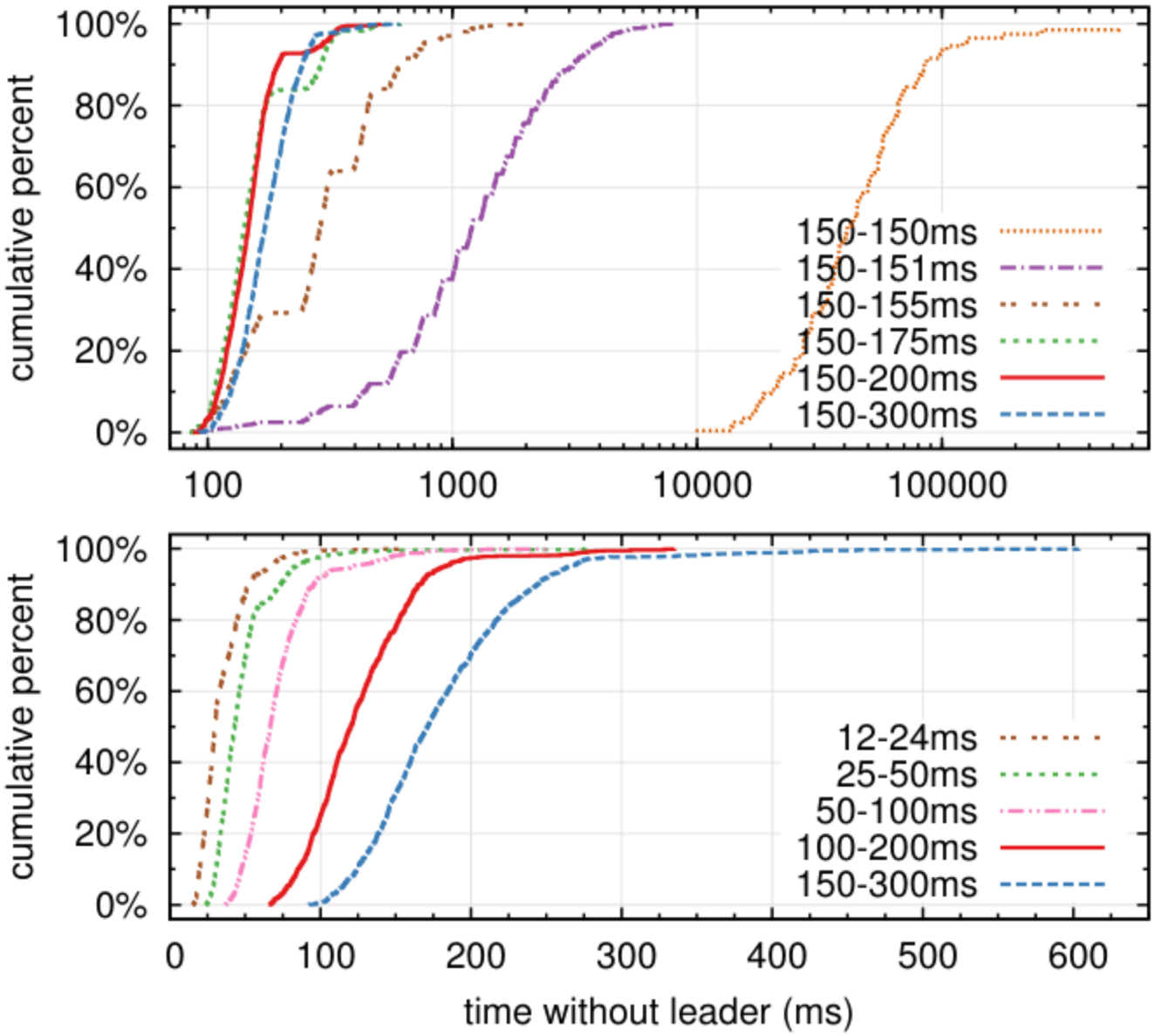


Рисунок 16: Время обнаружения и замены вышедшего из строя лидера. Верхний график изменяет степень случайности таймаутов выборов, а нижний график масштабирует минимальный таймаут выборов. Каждая строка представляет 1000 попыток (кроме 100 попыток для «150–150 мс») и соответствует определенному выбору таймаутов выборов; например, «150–155 мс» означает, что таймауты выборов выбирались случайным образом и равномерно в диапазоне от 150 мс до 155 мс. Измерения проводились на кластере из пяти серверов со временем трансляции примерно 15 мс. Результаты для кластера из девяти серверов аналогичны.

Верхний график на рисунке 16 показывает, что небольшой степени рандомизации таймаута выборов достаточно, чтобы избежать разделения голосов на выборах. В отсутствие случайности выбор лидера в наших тестах всегда занимал более 10 секунд из-за большого количества разделенных голосов. Добавление всего лишь 5 мс случайности существенно помогает, в результате чего среднее время простоя составляет 287 мс. Использование большей случайности улучшает поведение в худшем случае: при случайности 50 мс время завершения в худшем случае (более 1000 испытаний) составило 513 мс.

Нижний график на рисунке 16 показывает, что время простоя можно сократить, уменьшив время ожидания выборов. При таймауте выборов 12–24 мс на избрание лидера уходит в среднем всего 35 мс (самое продолжительное испытание заняло 152 мс). Однако уменьшение таймаутов сверх этого значения нарушает требования Raft к времени: лидеры испытывают трудности с рассылкой heartbeat-сообщений до того, как другие серверы начнут новые выборы. Это может привести к ненужной смене лидеров и снижению общей доступности системы. Мы рекомендуем использовать консервативный таймаут выборов, например 150–300 мс; такие таймауты вряд ли приведут к ненужной смене лидеров и по-прежнему обеспечат хорошую доступность.

10. Связанные работы

Было множество публикаций, посвященных алгоритмам консенсуса, многие из которых попадают в одну из следующих категорий:

- Исходное описание Лампорта Paxos [15] и попытка объяснить его более ясно [16, 20, 21].
- Разработки Paxos, которые заполняют недостающие детали и изменяют алгоритм, чтобы обеспечить лучшую основу для реализации [13, 26, 39].
- Системы, реализующие алгоритмы консенсуса, такие как Chubby [2, 4], ZooKeeper [11, 12] и Spanner [6]. Алгоритмы Chubby и Spanner подробно не публиковались, хотя оба утверждают, что основаны на Paxos. Алгоритм ZooKeeper был опубликован более подробно, но он сильно отличается от Paxos.
- Оптимизация производительности, которую можно применить к Paxos [1, 3, 18, 19, 25, 27].
- Viewstamped Replication (VR) Оки и Лискова, альтернативный подход к консенсусу, разработанный примерно в то же время, что и Paxos. Первоначальное описание [29] было переплетено с протоколом распределенных транзакций, но основной протокол консенсуса был выделен в недавнем обновлении [22]. VR использует подход, основанный на лидерстве, во многом схожий с Raft.

Самая большая разница между Raft и Paxos заключается в сильном лидерстве Raft: Raft использует выборы лидера как важную часть протокола консенсуса и концентрирует в лидере как можно больше функций. Такой подход приводит к более простому алгоритму, который легче понять. Например, в Paxos выборы лидера ортогональны базовому протоколу консенсуса: они служат только для оптимизации производительности и не требуются для достижения консенсуса. Однако это приводит к появлению дополнительного механизма: Paxos включает как двухфазный протокол для базового консенсуса, так и отдельный механизм для выборов лидера. Напротив, Raft включает выборы лидера непосредственно в алгоритм консенсуса и использует его как первую из двух фаз консенсуса. Это приводит к меньшему количеству механизмов, чем в Paxos.

Как и Raft, VR и ZooKeeper основаны на лидерах и поэтому имеют многие преимущества Raft перед Paxos. Однако у Raft меньше механизмов, чем у VR или ZooKeeper, поскольку он сводит к минимуму функциональность у нелидеров. Например, записи журнала в Raft движутся только в одном направлении: наружу от лидера в RPC **AppendEntries**. В журнале VR записи идут в обоих направлениях (лидеры могут получать записи журнала во время процесса выборов); это приводит к дополнительному механизму и сложности. Опубликованное описание ZooKeeper также передает записи журнала как лидеру, так и от него, но реализация, видимо, больше похожа на Raft [35].

Raft имеет меньше типов сообщений, чем любой другой известный нам алгоритм репликации журналов на основе консенсуса. Например, мы подсчитали типы сообщений, которые VR и ZooKeeper используют для базового консенсуса и изменений членства (исключая сжатие журналов и взаимодействие с клиентом, поскольку они практически не зависят от алгоритмов). VR и ZooKeeper определяют по 10 различных типов сообщений, тогда как Raft имеет только 4 типа сообщений (два запроса RPC и их ответы). Сообщения Raft немного более плотны, чем сообщения других алгоритмов, но в совокупности они проще. Кроме того, VR и ZooKeeper описаны с точки зрения передачи целых логов при смене лидера; дополнительные типы сообщений потребуются для оптимизации этих механизмов, чтобы они были практичными.

Сильный лидерский подход Raft упрощает алгоритм, но исключает некоторую оптимизацию производительности. Например, эгалитарист Paxos (EPaxos) может при некоторых условиях добиться

более высокой производительности с помощью подхода без лидера [27]. EPaxos использует коммутативность в командах конечного автомата. Любой сервер может передать команду всего за один раунд связи, при условии, что другие предложенные команды коммутируют с ним одновременно. Однако, если команды, предлагаемые одновременно, не коммутируются друг с другом, EPaxos требует дополнительного раунда связи. Поскольку любой сервер может фиксировать команды, EPaxos хорошо балансирует нагрузку между серверами и может достичь более низкой задержки, чем Raft, в настройках WAN. Однако это существенно усложняет Paxos.

Несколько различных подходов к изменению членства в кластере были предложены или реализованы в других работах, включая исходное предложение Лэмпорта [15], VR [22] и SMART [24]. Мы выбрали подход совместного консенсуса для Raft, поскольку он использует остальную часть протокола консенсуса, поэтому для изменения членства требуется очень мало дополнительных механизмов. Подход Лампорта, основанный на \alpha, не подходил для Raft, поскольку он предполагает, что консенсус может быть достигнут без лидера. По сравнению с VR и SMART алгоритм реконфигурации Raft имеет то преимущество, что изменения членства могут происходить без ограничения обработки обычных запросов; напротив, VR останавливает всю нормальную обработку во время изменения конфигурации, а SMART накладывает ограничение на количество необработанных запросов, подобное \alpha. Подход Raft также добавляет меньше механизмов, чем VR или SMART.

11. Заключение

Алгоритмы часто разрабатываются с учетом корректности, эффективности и/или краткости в качестве основных целей. Хотя все это достойные цели, мы считаем, что понятность не менее важна. Ни одна из других целей не может быть достигнута до тех пор, пока разработчики не доведут алгоритм до практической реализации, которая неизбежно будет отклоняться от опубликованной формы и расширяться. Если разработчики не имеют глубокого понимания алгоритма и не могут составить о нем интуитивное представление, им будет трудно сохранить его желаемые свойства в своей реализации.

В этой статье мы рассмотрели проблему распределенного консенсуса, где широко распространенный, но непонятный алгоритм Paxos на протяжении многих лет бросал вызов студентам и разработчикам. Мы разработали новый алгоритм Raft, который, как мы показали, более понятен, чем Paxos. Мы также считаем, что Raft обеспечивает лучшую основу для построения системы. Использование понятности в качестве основной цели проектирования изменило наш подход к проектированию Raft; По мере разработки мы обнаружили, что неоднократно использовали некоторые методы, такие как декомпозиция проблемы и упрощение пространства состояний. Эти приемы не только улучшили понимание Raft, но и облегчили уверенность в его правильности.

12. Благодарности

Исследование пользователей было бы невозможно без поддержки Али Годси, Дэвида Мазьера и студентов CS 294-91 в Беркли и CS 240 в Стэнфорде. Скотт Клеммер помог нам разработать исследование пользователей, а Нельсон Рэй консультировал нас по статистическому анализу. Слайды Paxos для исследования с участием пользователей во многом заимствованы из набора слайдов, первоначально созданного Лоренцо Альвизи. Особая благодарность выражается Дэвиду Мазьеру и Эзре Хоху за обнаружение тонких ошибок в Raft. Многие люди предоставили полезные отзывы о статье и материалах исследования, в том числе Эд Буньон, Майкл Чан, Хьюз Эврар, Дэниел Гиффин, Арджун Гопалан, Джон Хауэлл, Вималкумар Джеякумар, Анкита Кеджривал, Александр Кракун, Амит Леви, Джоэл Мартин, Сатоши Мацусита, Олег Песок, Дэвид Рамос, Робберт ван Ренесс, Мендель Розенблюм, Николас Шипер, Деян Стефан, Эндрю Стоун, Райан Статсман, Дэвид Терей, Стивен Янг, Матей Захария, 24 анонимных рецензента конференции (с учётом повторных рецензий) и особенно наш куратор статьи Эдди Колер. Вернер Фогельс опубликовал в Твиттере ссылку на более ранний проект, который предоставил Raft значительную известность. Эта работа поддерживалась Исследовательским центром гигамасштабных систем и Центром многомасштабных систем, двумя из шести исследовательских центров, финансируемых в рамках Исследовательской программы фокус-центра, программы Корпорации полупроводниковых исследований, STARnet, программы Корпорации полупроводниковых исследований, спонсируемой MARCO и DARPA, Национального научного фонда в рамках гранта № 0963859, а также грантов Facebook, Google, Mellanox, NEC, NetApp, SAP и Samsung. Диего Онгаро поддерживается Стэнфордской стипендией для выпускников Junglee Corporation.

Литература

[1] BOLOSKY, W. J., BRADSHAW, D., HAAGENS, R. B., KUSTERS, N. P., AND LI, P. Paxos replicated state machines as the basis of a high-performance data store. In *Proc. NSDI'11, USENIX Conference on Networked Systems Design and Implementation* (2011), USENIX, pp. 141–154.

[2] BURROWS, M. The Chubby lock service for loosely-coupled distributed systems. In *Proc. OSDI'06, Symposium on Operating Systems Design and Implementation* (2006), USENIX, pp. 335–350.

[3] CAMARGOS, L. J., SCHMIDT, R. M., AND PEDONE, F. Multicoordinated Paxos. In *Proc. PODC'07, ACM Symposium on Principles of Distributed Computing* (2007), ACM, pp. 316–317.

[4] CHANDRA, T. D., GRIESEMER, R., AND REDSTONE, J. Paxos made live: an engineering perspective. In *Proc. PODC'07, ACM Symposium on Principles of Distributed Computing* (2007), ACM, pp. 398–407.

[5] CHANG, F., DEAN, J., GHEMAWAT, S., HSIEH, W. C., WALLACH, D. A., BURROWS, M., CHANDRA, T., FIKES, A., AND GRUBER, R. E. BigTable: a distributed storage system for structured data. In *Proc. OSDI'06, USENIX Symposium on Operating Systems Design and Implementation* (2006), USENIX, pp. 205–218.

[6] CORBETT, J. C., DEAN, J., EPSTEIN, M., FIKES, A., FROST, C., FURMAN, J. J., GHEMAWAT, S., GUBAREV, A., HEISER, C., HOCHSCHILD, P., HSIEH, W., KANTHAK, S., KOGAN, E., LI, H., LLOYD, A., MELNIK, S., MWAURA, D., NAGLE, D., QUINLAN, S., RAO, R., ROLIG, L., SAITO, Y., SZYMANIAK, M., TAYLOR, C., WANG, R., AND WOODFORD, D. Spanner: Google's globally-distributed database. In *Proc. OSDI'12, USENIX Conference on Operating Systems Design and Implementation* (2012), USENIX, pp. 251–264.

[7] COUSINEAU, D., DOLIGEZ, D., LAMPORT, L., MERZ, S., RICKETTS, D., AND VANZETTO, H. TLA + proofs. In *Proc. FM'12, Symposium on Formal Methods* (2012), D. Giannakopoulou and D. Méry, Eds., vol. 7436 of *Lecture Notes in Computer Science* , Springer, pp. 147–154.

[8] GHEMAWAT, S., GOBIOFF, H., AND LEUNG, S.-T. The Google file system. In *Proc. SOSP'03, ACM Symposium on Operating Systems Principles* (2003), ACM, pp. 29–43.

[9] GRAY, C., AND CHERITON, D. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the 12th ACM Symposium on Operating Systems Principles* (1989), pp. 202–210.

[10] HERLIHY, M. P., AND WING, J. M. Linearizability: a correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems 12* (July 1990), 463–492.

[11] HUNT, P., KONAR, M., JUNQUEIRA, F. P., AND REED, B. ZooKeeper: wait-free coordination for internet-scale systems. In *Proc ATC'10, USENIX Annual Technical Conference* (2010), USENIX, pp. 145–158.

[12] JUNQUEIRA, F. P., REED, B. C., AND SERAFINI, M. Zab: High-performance broadcast for primary-backup systems. In *Proc. DSN'11, IEEE/IFIP Int'l Conf. on Dependable Systems & Networks* (2011), IEEE Computer Society, pp. 245–256.

[13] KIRSCH, J., AND AMIR, Y. Paxos for system builders. Tech. Rep. CNDS-2008-2, Johns Hopkins University, 2008.

[14] LAMPORT, L. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM 21* , 7 (July 1978), 558–565.

[15] LAMPORT, L. The part-time parliament. *ACM Transactions on Computer Systems 16* , 2 (May 1998), 133–169.

[16] LAMPORT, L. Paxos made simple. *ACM SIGACT News 32* , 4 (Dec. 2001), 18–25.

[17] LAMPORT, L. *Specifying Systems, The TLA+ Language and Tools for Hardware and Software Engineers* . Addison-Wesley, 2002.

[18] LAMPORT, L. Generalized consensus and Paxos. Tech. Rep. MSR-TR-2005-33, Microsoft Research, 2005.

[19] LAMPORT, L. Fast paxos. *Distributed Computing 19* , 2 (2006), 79–103.

[20] LAMPSON, B. W. How to build a highly available system using consensus. In *Distributed Algorithms* , O. Baboaglu and K. Marzullo, Eds. Springer-Verlag, 1996, pp. 1–17.

[21] LAMPSON, B. W. The ABCD's of Paxos. In *Proc. PODC'01, ACM Symposium on Principles of Distributed Computing* (2001), ACM, pp. 13–13.

[22] LISKOV, B., AND COWLING, J. Viewstamped replication revisited. Tech. Rep. MIT-CSAIL-TR-2012-021, MIT, July 2012.

[23] LogCabin source code. http://github.com/logcabin/logcabin.

[24] LORCH, J. R., ADYA, A., BOLOSKY, W. J., CHAIKEN, R., DOUCEUR, J. R., AND HOWELL, J. The SMART way to migrate replicated stateful services. In *Proc. EuroSys'06, ACM SIGOPS European Conference on Computer Systems* (2006), ACM, pp. 103–115.

[25] MAO, Y., JUNQUEIRA, F. P., AND MARZULLO, K. Mencius: building efficient replicated state machines for WANs. In *Proc. OSDI'08, USENIX Conference on Operating Systems Design and Implementation* (2008), USENIX, pp. 369–384.

[26] MAZIÈRES, D. Paxos made practical. http://www.scs.stanford.edu/~dm/home/papers/paxos.pdf, Jan. 2007.

[27] MORARU, I., ANDERSEN, D. G., AND KAMINSKY, M. There is more consensus in egalitarian parliaments. In *Proc. SOSP'13, ACM Symposium on Operating System Principles* (2013), ACM.

[28] Raft user study. http://ramcloud.stanford.edu/~ongaro/userstudy/.

[29] OKI, B. M., AND LISKOV, B. H. Viewstamped replication: A new primary copy method to support highly-available distributed systems. In *Proc. PODC'88, ACM Symposium on Principles of Distributed Computing* (1988), ACM, pp. 8–17.

[30] O'NEIL, P., CHENG, E., GAWLICK, D., AND ONEIL, E. The log-structured merge-tree (LSM-tree). *Acta Informatica 33* , 4 (1996), 351–385.

[31] ONGARO, D. *Consensus: Bridging Theory and Practice* . PhD thesis, Stanford University, 2014 (work in progress). http://ramcloud.stanford.edu/~ongaro/thesis.pdf.

[32] ONGARO, D., AND OUSTERHOUT, J. In search of an understandable consensus algorithm. In *Proc ATC'14, USENIX Annual Technical Conference* (2014), USENIX.

[33] OUSTERHOUT, J., AGRAWAL, P., ERICKSON, D., KOZYRAKIS, C., LEVERICH, J., MAZIÈRES, D., MITRA, S., NARAYANAN, A., ONGARO, D., PARULKAR, G., ROSENBLUM, M., RUMBLE, S. M., STRATMANN, E., AND STUTSMAN, R. The case for RAMCloud. *Communications of the ACM 54* (July 2011), 121–130.

[34] Raft consensus algorithm website. https://raft.github.io.

[35] REED, B. Personal communications, May 17, 2013.

[36] ROSENBLUM, M., AND OUSTERHOUT, J. K. The design and implementation of a log-structured file system. *ACM Trans. Comput. Syst. 10* (February 1992), 26–52.

[37] SCHNEIDER, F. B. Implementing fault-tolerant services using the state machine approach: a tutorial. *ACM Computing Surveys 22* , 4 (Dec. 1990), 299–319.

[38] SHVACHKO, K., KUANG, H., RADIA, S., AND CHANSLER, R. The Hadoop distributed file system. In *Proc. MSST'10, Symposium on Mass Storage Systems and Technologies* (2010), IEEE Computer Society, pp. 1–10.

[39] VAN RENESSE, R. Paxos made moderately complex. Tech. Rep., Cornell University, 2012.